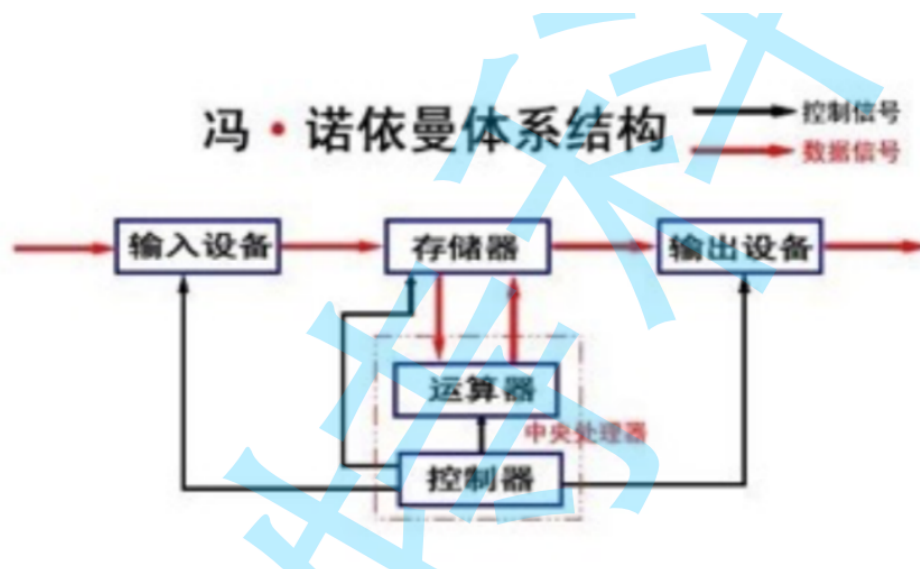


BIT.3_Linux进程概念.pdf

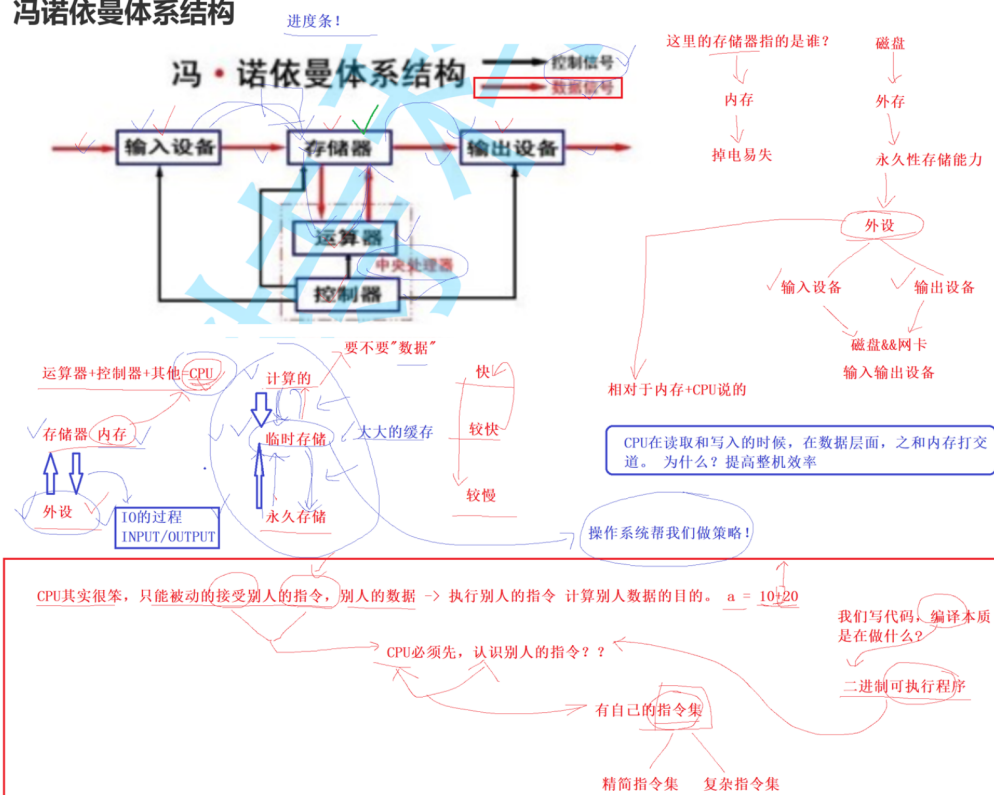
1冯诺依曼体系结构

1.1 体系结构

输入设备，输出设备，存储器(内存设备)，中央处理器（CPU）。四大件。



冯诺依曼体系结构



这里的存储器是？内存。内存掉电易失

磁盘，外设，永久性存储能力。

外设：输入设备

外设：输出设备

磁盘和网卡 既是输入设备也是输出设备

为什么是外设？相对于：内存+CPU

运算器+控制器+其它=CPU	存储器 内存	外设
很快	快	慢
计算的	临时存储的	永久性存储

CPU其实很笨，只能被动接受别人的指令，别人的数据-->执行别人的指令，计算别人数据的目的。

CPU必须先，认识别人的指令？CPU有自己的指令集。我们写代码，编译本质-->二进制可执行程序。实际上就是翻译成CPU的指令集。

所以编译器存在的意义，翻译我们的程序到机器指令。

CPU在读取和写入的时候，在数据层面，只和内存打交道。为了提高整机效率。

1.2 内存

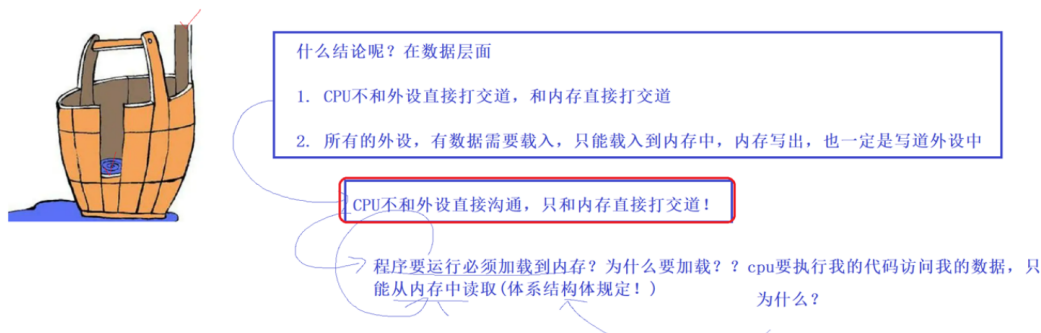
CPU里面的数据，不是天然就存在的。从外设拿数据那就太慢了。所以综合考虑就从内存拿数据。

CPU读取和写入只和内存打交道，提高整机效率。

内存里面不是天然就有数据的

例如：开机的时候加载 操作系统加载到内存。

内存可以看做一个大大的缓存，匹配CPU和外设的速度。



OS操作系统---》帮我们做决策。

操作系统管理 IO过程。

内存

I/O 读写就是IO的过程。

外设

总结

CPU不和外设直接打交道，只是和内存直接打交道。

所以的外设，有数据需要载入，只能载入到内存中，内存写出，也一定是写到外设中。

总结：----》》CPU不和外设直接打交道，只和内存直接打交道。

1.3 数据传输

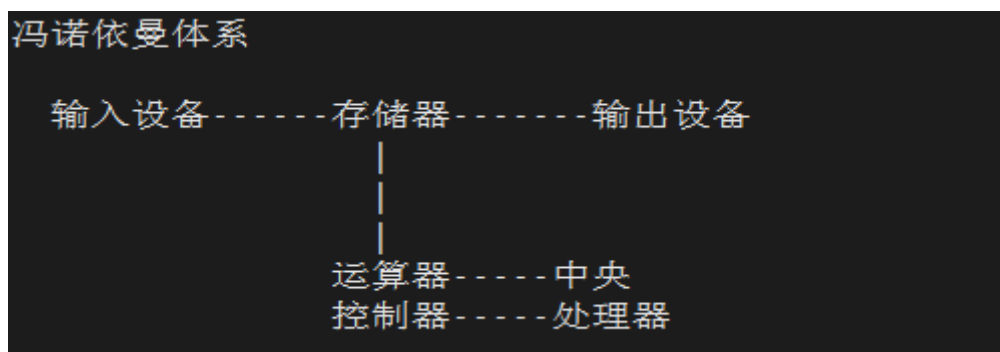
程序要运行必须加载到内存？

为什么要加载到内存？

CPU要执行我的代码，访问我的数据。

只能从内存中读取。(体系结构规定的){为什么这样子规定，效率的问题}

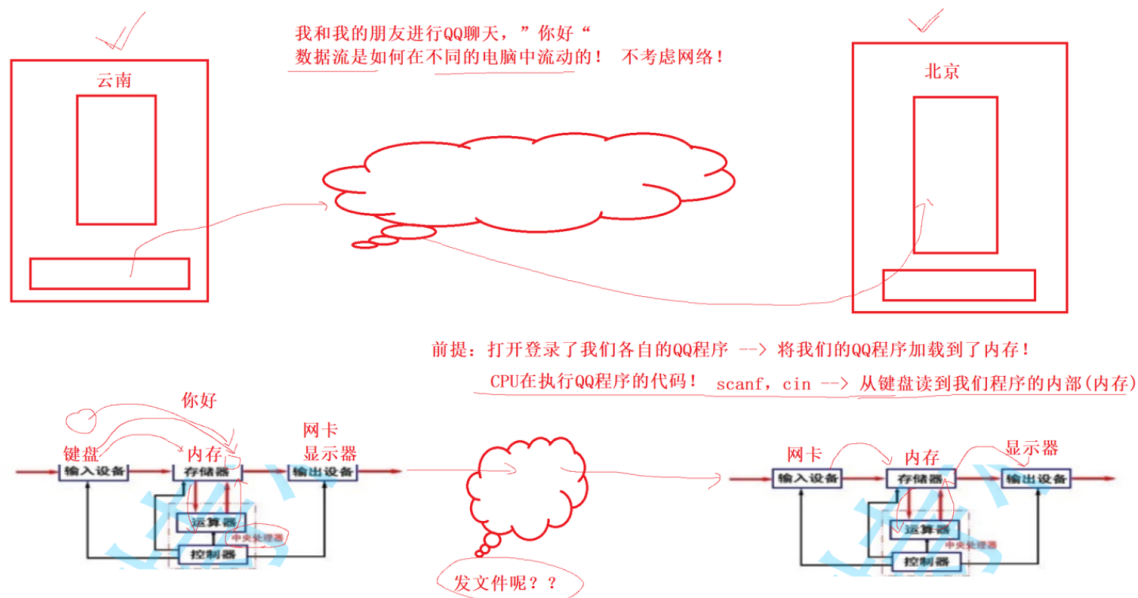
进度条：数据写到内存里面---->没有及时刷新到外设。



两个朋友发消息，冯诺依曼体系结构

计算器1	计算器2
键盘输入	网卡输入
输入到内存	输入到内存
CPU读取	输入到CPU
网卡和显示器输出	输入到显示器

question:
数据加密/解密。
登录了QQ----->将QQ加载达到内存里面。
CPU在执行QQ程序的代码！ cin,cout --> 从键盘读取数据到我们的程序的内部(内存)。



发文件也是类似的理解

1.4 总结

截至目前，我们所认识的计算机，都是有一个个的硬件组件组成

- 输入单元：包括键盘，鼠标，扫描仪，写板等
- 中央处理器(CPU)：含有运算器和控制器等
- 输出单元：显示器，打印机等

关于冯诺依曼，必须强调几点：

- 这里的存储器指的是内存
- 不考虑缓存情况，这里的CPU能且只能对内存进行读写，不能访问外设(输入或输出设备)
- 外设(输入或输出设备)要输入或者输出数据，也只能写入内存或者从内存中读取。
- 一句话，所有设备都只能直接和内存打交道。

2操作系统(OS)

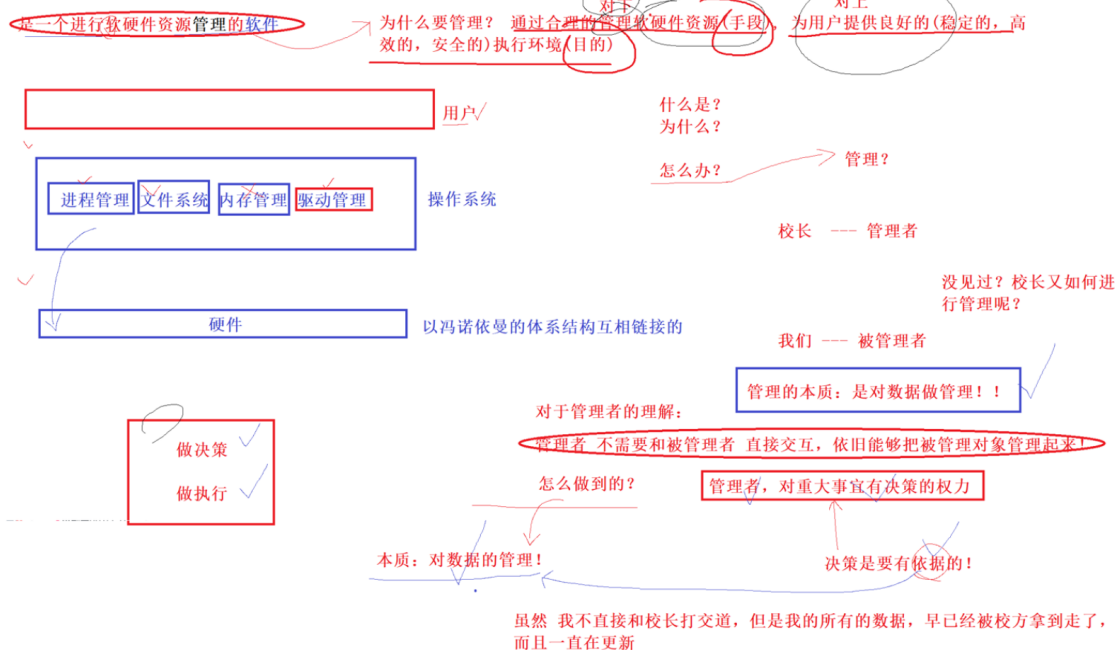
2.1 建模

OS是一个进行软硬件的资源管理的软件。

它属于软件，但职责是直接控制硬件、向上屏蔽硬件细节。

操作系统

录制中03:05:45



用户
用户接口
系统接口
进程管理，文件系统管理，内存管理，驱动管理。（os）
驱动
硬件

OS为什么要进行管理呢？

通过合理的管理软硬件资源，为用户提供良好的执行环境。良好：稳定，安全，高效的。

是什么？：OS是一个进行软硬件的资源管理的软件

为什么？：通过合理的管理软硬件资源，为用户提供良好的执行环境。良好：稳定，安全，高效的

怎么办？：管理

举个例子：理解管理者

校长：管理者，决策权利

辅导员：驱动齿，执行权利

我们：被管理者

管理：管理不需要面对面，只需要拿到被管理者的数据。对重大事情有决策的权力。

做决策需要依据的，依据就是根据数据。管理的本质就是对数据做管理。

管理的本质是对:被管理者的数据做管理。

管理者，对重大事有决策的能力。决策的依据就是数据。

管理者： 做决策

校长 OS(操作系统) 软件

执行者： 做执行 1.执行上面的命令 2.和被管理者接触，拿到相应的数据。

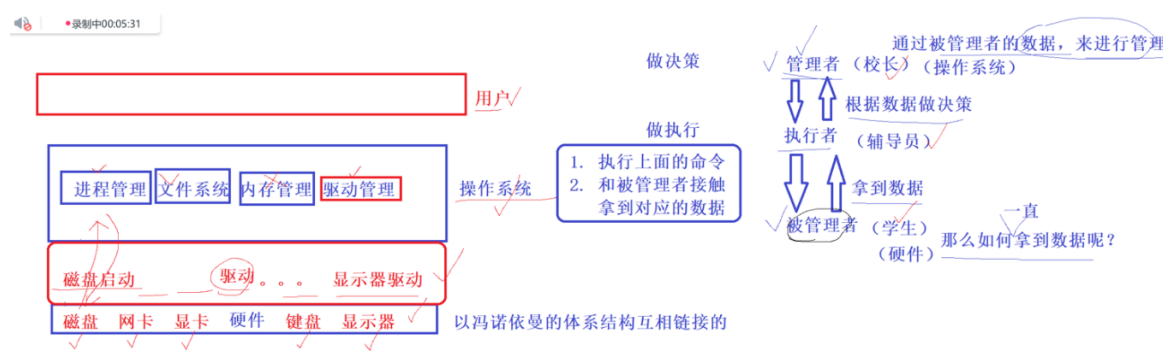
辅导员 驱动 (磁盘驱

动，显示器驱动) 软件

被管理者： 行动产生数据

学生 硬件 (磁盘，网卡，

显卡，键盘，显示器) 硬件



2.2 先描述，后组织

描述的本质就是对一个对象进行管理。组织就是对多个对象进行管理的。

被管理者的数据太多了，怎么办？

学生的信息种类都是一样的。

被管理对象进行建模的过程！

```
struct stu
{
    name;
    sex;
    address;
    telephone;
    qq;
    score;
    struct stu* next;
};
```

```
struct stu zhangsan = {};
struct stu lisi = {};
struct stu wangwu = {};
```

转换成对链表的管理

```
head-->stu1-->stu2-->stu3-->null
```

对学生做管理，变成对链表数据做管理。

先描述：抽象数据。c结构体，c++类

后组织：数据结构。数据结构组织起来

所有的管理的本质，先描述在组织。

1. 通讯录管理 a. 先构建结构体 b. 数据结构组织。

2. 先写一个类(先描述)，list, vector(组织)

OS管理硬件：

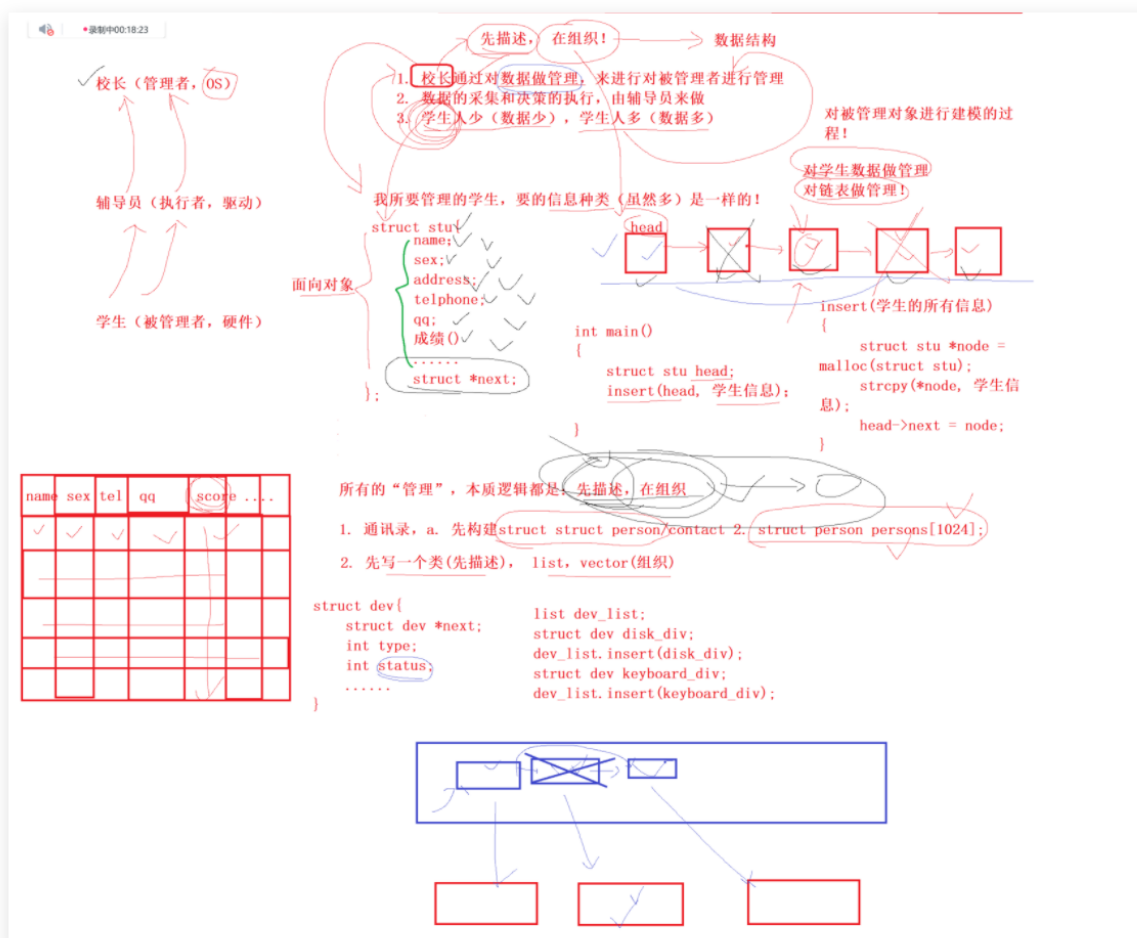
struct dev {struct dev* next, int type, int status}

管理的本质：对数据做管理

管理的方法：先描述（语言），在组织 数据结构

OS管理硬件：一模一样的

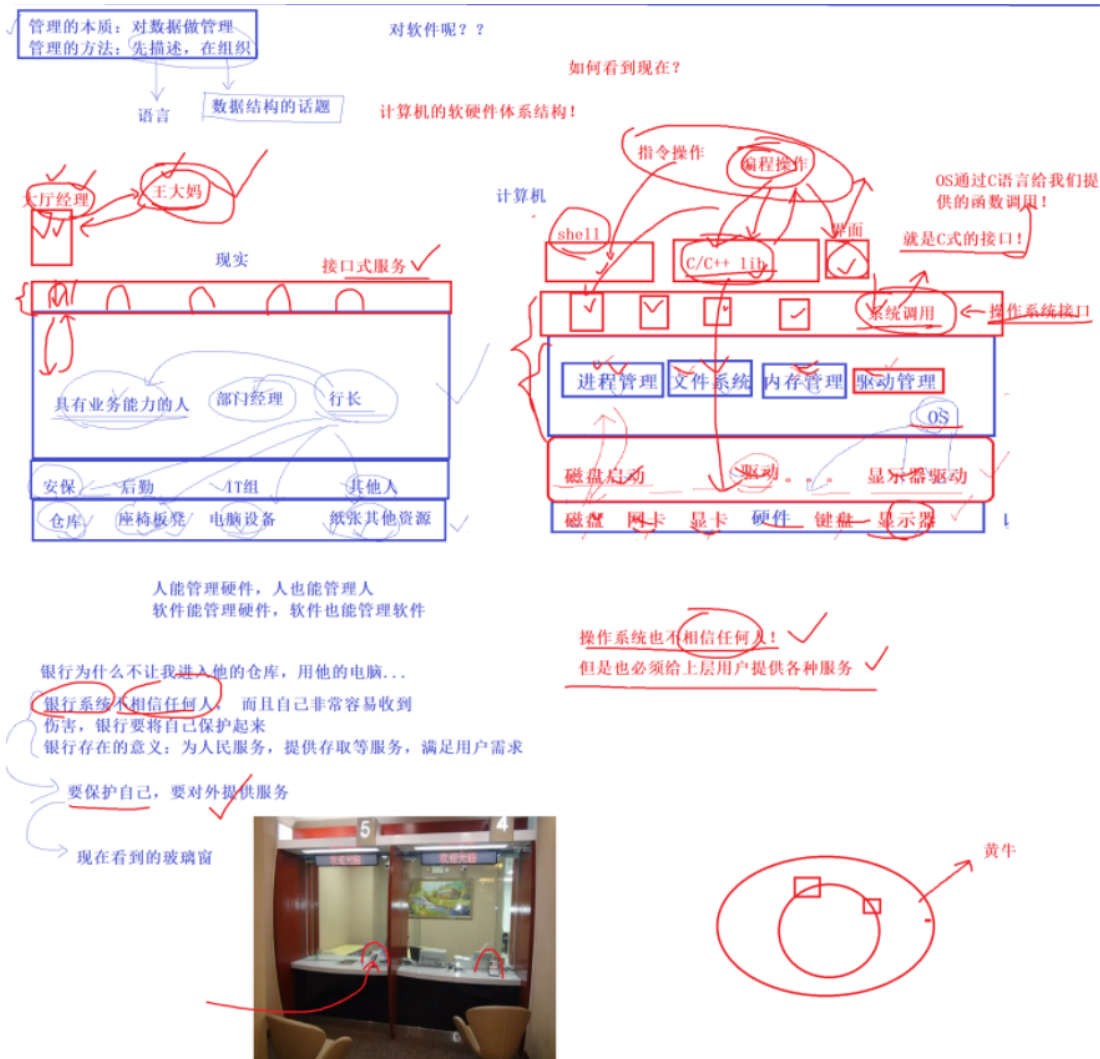
软件管理软件，软件管理硬件。



2.3 OS

OS需要保护自己的同时，还要为用户提供服务。

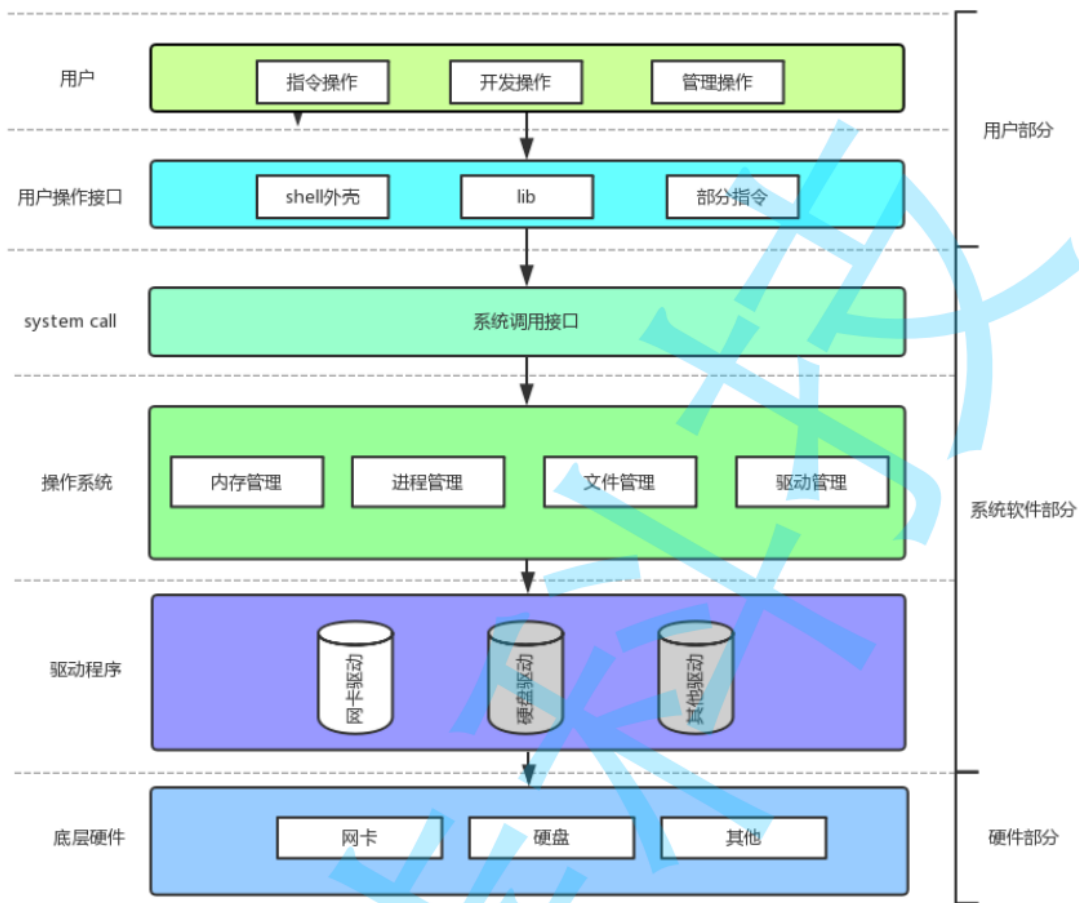
shell指令，c/c++ lib 编程操作。



OS不相信任何人

- 1.保护自己， 不让任何人直接进入操作系统
- 2.对外提供服务，系统调用接口

计算机的软硬件体系结构
用户
系统调用接口(系统调用) (本质上就是c式的接口) (man syscalls) (strace ls) 实际上就是函数接口
(shell(指令操作)/lib(c/c++)(编程操作)/界面)
操作系统(进程控制，文件系统，内存管理，驱动管理)
驱动
硬件



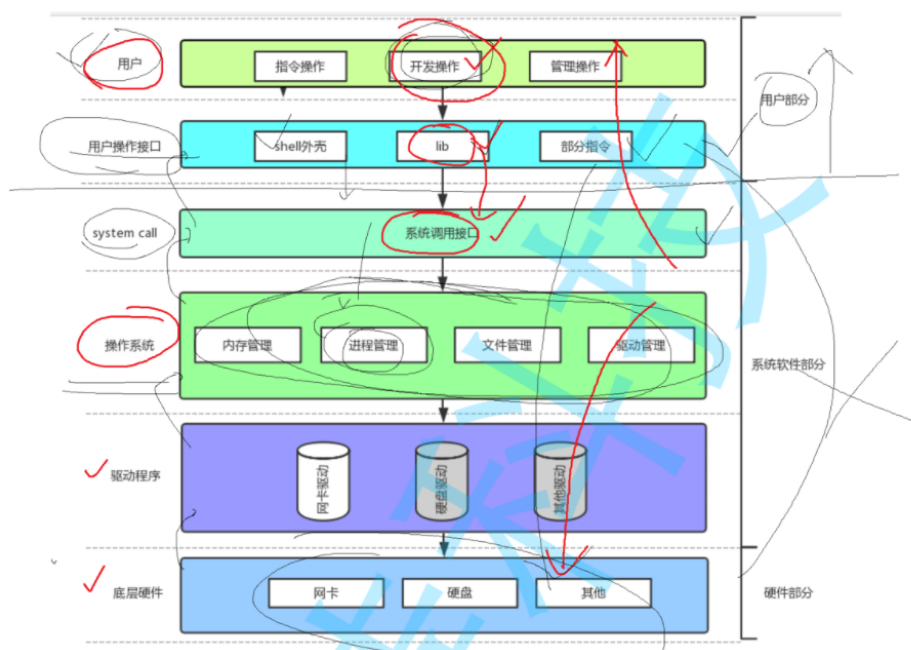
计算机软硬件体系结构

我们学的c/c++: 学习用户接口和系统调用的接口。

1.冯诺依曼体系

2.操作系统本身理解

3.计算机体系结构



3进程

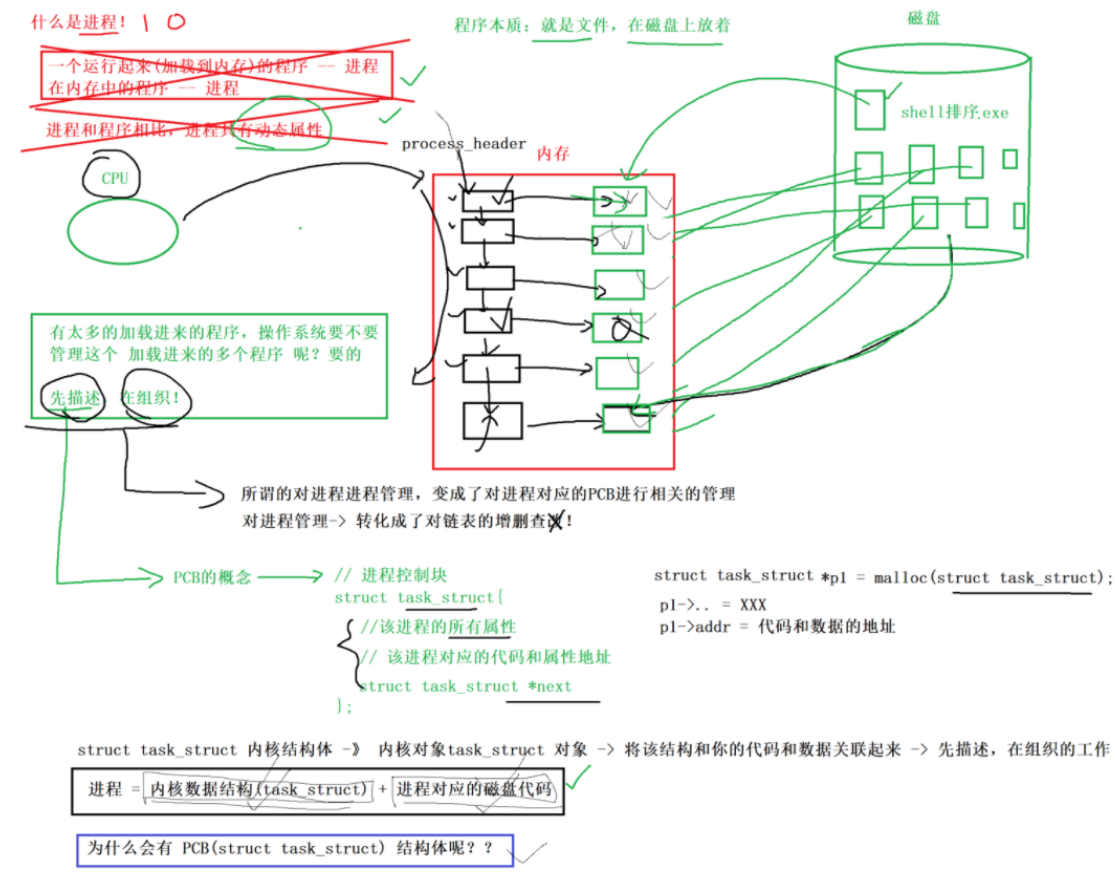
3.1进程

进程 == 内核数据结构 + 进程对应的磁盘代码

那么多的进程需要被管理起来，如何管理呢？进程控制管。

一个运行起来(加载到内存)的程序---进程-- 一般概念

在内存中的程序的-----进程



写好的程序本质就是文件，在磁盘上放着。

那么多的程序，操作系统如何管理呢？

先描述:PCB。进程控制块。Process control block

后组织:

Linux 的 PCB 之间通过 多重链表 + 树形结构 连接，形成复杂但高效的进程组织体系。
所以——它不仅仅是一个链表，而是多个链表构成的网络结构。

```
struct task_struct
{
    pid_t pid;           // 进程号
    long state;          // 进程状态
    int prio;             // 优先级
    struct mm_struct *mm; // 地址空间信息
}
```

```

struct files_struct *files;    // 打开的文件信息
struct signal_struct *signal; // 信号处理相关
struct task_struct *parent;   // 父进程
struct list_head children;    // 子进程链表
struct list_head tasks;       // 系统全局进程链表节点
struct thread_struct thread;   // CPU 寄存器等上下文信息
struct sched_entity se;       // 调度实体信息
};

```

所谓对进程进行管，变成了对进程对应的PCB进行相关的管理。

对进程的管理-->对链表的增删查改

struct task_struct 内核结构体 --> 内核结构体的对象 ---> 将该结构体和你的代码和数据关联起来----> 先描述，后组织。

进程 == 内核数据结构 + 进程对应的磁盘代码

为什么会有PCB(struct task_struct)的结构体呢？

进程在调度运行的时候，就具有动态属性。创建一个进程，就要创建一个struct task_struct{}

查看当前终端的pid

已思考 (用时 2 秒)

- 我们有一个问题: "查看当前终端的pid"。需要解释如何查看当前终端的进程ID。可能是在Linux/Unix环境下。常用的方法: 使用 `echo $$` 输出当前shell的pid。或者使用 `ps` 命令结合终端名。也可以使用 `tty` 命令查看当前终端设备, 然后 `ps -t`。但最简单的就是 `echo $$`。用户可能期望一个命令行指令。回答应该简洁明确。

要查看当前终端的进程 ID (PID), 可以在终端中执行以下命令:

bash

echo \$\$

这会输出当前 shell 的 PID, 也就是你所使用的终端进程的 ID。

另外, 也可以使用 `$PPID` 查看父进程 ID, 或者使用 `ps -t $(tty)` 查看当前终端下运行的进程列表。

`ps -axj | head -1 && ps ajx | grep a.out`

`getpid()`

linux进程也是文件的。

`/proc/id`: 进程目录文件 (了解的)


```

#include <unistd.h>
#include <sys/types.h>
#include <assert.h>

int main()
{
    pid_t id = fork();
    assert(id >= 0);

    if(id == 0)
    {
        while(1)
        {
            printf("I AM PORCESS, ppid : %d, pid : %d id : %d \n",
                getppid(),
                getpid(),
                id);
            sleep(2);
        }
    }
    else
    {
        while(1)
        {
            printf("I AM PORCESS, ppid : %d, pid : %d id : %d \n",
                getppid(),
                getpid(),
                id);
            sleep(2);
        }
    }
    return 0;
}

```

```

[root@lic lesson48]# ./a.out
I AM PARENT PORCESS, ppid : 2025, pid : 3314
I AM CHILD PORCESS, ppid : 3314, pid : 3315
I AM PARENT PORCESS, ppid : 2025, pid : 3314
I AM CHILD PORCESS, ppid : 3314, pid : 3315
^C
[root@lic lesson48]# █

```

进程在调度运行的时候，进程就具有动态属性！

kill -9 进程id

见见与进程有关的系统调用

getpid();

✓ 另一种查看进程的方式 了解 ✓

学习常见进程调用 ✓ 录制中02:59:56 ✓

命令行上启动的进程，一般它的父进程没有特殊情况的话，都是bash！

创建子进程 -- fork 是一个函数 -- 函数执行前：只有一个父进程 -- 函数执行后：父进程+子进程

{ fork() 之后，会有父进程+子进程两个进程在执行后续代码
fork 后续的代码，被父子进程共享！
通过返回值不同，让父子进程执行后续共享的代码的一部分！

3.2 code

进程id和 fork

```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
int main()
{
    #if 1
        printf("当前bash的id:          %d\n", getppid());
        printf("当前bash的子进程的id:%d\n", getpid());
    #else
        // fork之后代码共享
        // 同一个变量，在不被修改的情况下，竟然有不同的内容

        int a = 1000;
        // 前面的代码，只有父进程
        pid_t id = fork();
        if(id == -1)
        {
            perror("fork");
            return 1;
        }
        // printf("我是一个进程 我的id:%d，我的父进程的id: %d , %d \n", getpid(),
        // getpid(),id);

        // 后面父子进程都有，只要有共享代码修改，就会发生写实拷贝
        while(1)
        {
            if(id == 0)
            {
                a = 0;
                printf("我是子进程 我的pid:%d, a我的id: %d, a : %d, a的地址 : %p\n",
                    getppid(),
                    getpid(),
                    a,
                    &a);
            }
        }
    }
}
```

```

        sleep(2);
    }
    else if(id > 0)
    {
        a = 1;
        printf("我是父进程 我的pid:%d, 我的id: %d, a : %d, a的地址 : %p\n",
                getppid(),
                getpid(),
                a,
                &a);
        sleep(2);
    }
}

printf("zzz\n");

// 父进程的pid就是当前终端
// echo $$ 当前终端的PID

// 通过fork的返回值进行分流
#endif
return 0;
}

```

3.3 fork

fork之后代码共享

谁先被调度，不知道的。

父进程返回 子进程pid

子进程返回 0

父进程需要知道孩子是谁，所以返回孩子的 pid。

知道孩子的pid，方便后面的 回收工作的。

子进程拿到的是“我就是孩子”

4进程状态

4.1 复习

4.1.1 冯诺依曼体系

冯诺依曼体系：CPU只和内存打交道。外设也是只和内存打交道。缓存。

计算机是软件和硬件高度配合的

读 输入----内存----CPU

写 CPU-----内存----输出

CPU只能被动接收数据，指令。

CPU已经内置了指令。

二进制可执行程序---CUP认识的指令。（c++代码的编译链接信息。）

控制CPU执行的

管理的本质：对数据进行管理。

OS：是一个软硬件资源管理的软件。

OS对下软硬件资源

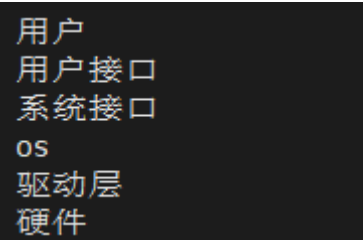
OS对上为用户提供良好的执行环境。

管理的本质就是：先描述，后组织。

管理者和被管理者不一定要接触。

管理的本质就是管理数据。

执行者：1被管理者的数据交给管理者 2执行指令。



管理的本质：先描述，后组织

先描述数据：结构体，类和对象。抽象出来

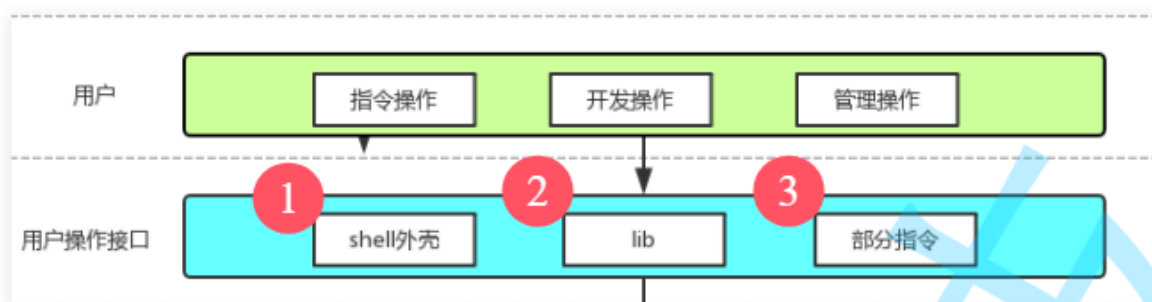
后组织数据：数据结构。链接起来的

OS维护一个数据结构----维护对应的对象。 c struct , cpp class

4.1.2 OS

OS不相信任何人。

shell, c/c++ lib ,界面, 指令集合



进程---操作系统里面的数据只能通过系统接口访问

什么是进程？

程序就是一段二进制代码,就是一个文件

描述进程的PCB

管理每一个PCB，用数据结构组织起来。

程序加载带内存，操作系统瞬间创建一个PCB

PCB里面有进程的所有信息。描述进程的全部信息。

进程 == 内核数据结构（PCB） + 进程对应的磁盘代码

PCB

- 标示符: 描述本进程的唯一标示符，用来区别其他进程。
- 状态: 任务状态，退出代码，退出信号等。
- 优先级: 相对于其他进程的优先级。
- 程序计数器: 程序中即将被执行的下一条指令的地址。
- 内存指针: 包括程序代码和进程相关数据的指针，还有和其他进程共享的内存块的指针
- 上下文数据: 进程执行时处理器的寄存器中的数据[休学例子，要加图CPU，寄存器]。
- I / O状态信息: 包括显示的I/O请求,分配给进程的I / O设备和被进程使用的文件列表。
- 记账信息: 可能包括处理器时间总和，使用的时钟数总和，时间限制，记账号等。
- 其他信息

4.2 进程状态

fork创建子进程

进程状态

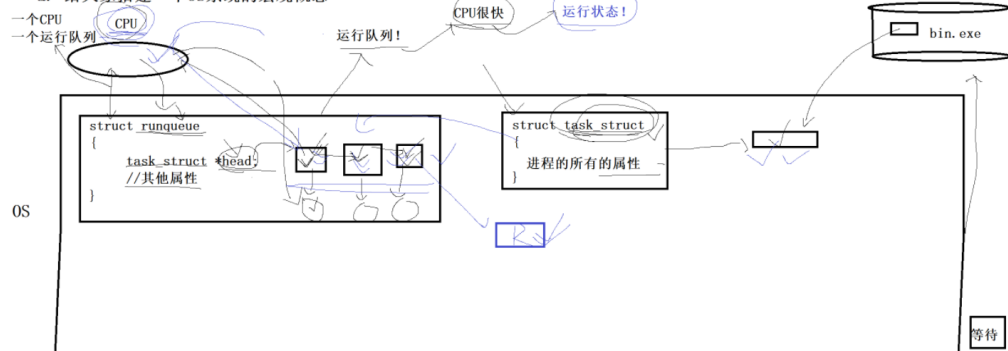
进程状态

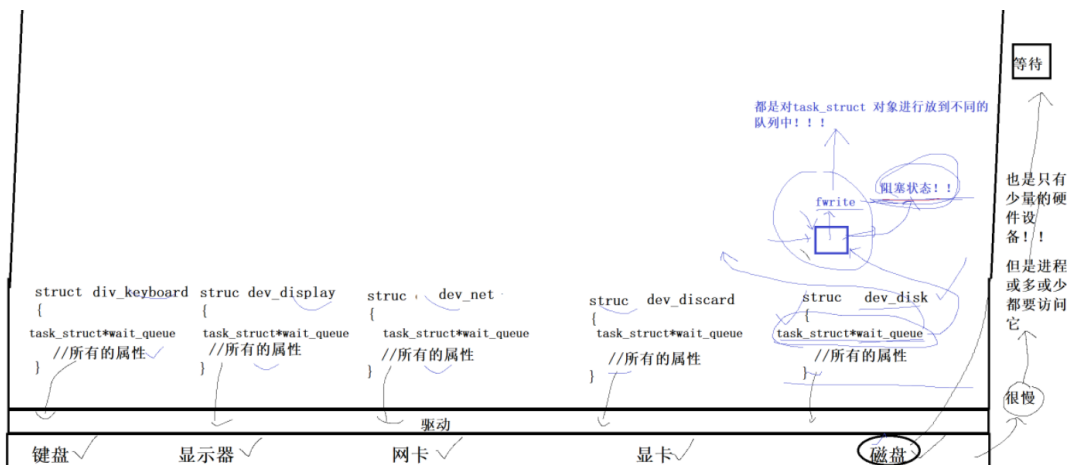
- 1.运行
- 2.新建
- 3.就绪
- 4.挂起
- 5.阻塞
- 6.等待
- 7.停止
- 8.挂机
- 9.死亡

Linux 内核中主要状态定义在 `task_struct->state` 中。

都是操作系统的说法，太抽象了，不是具体的一款操作系统。

1. 普遍的操作系统层面如何理解上面的概念
a. 给大家搭建一个os系统的宏观概念





1. 一个CPU一个运行队列
 2. 让进程入队列, 本质: 将该进程的task_struct 结构体对象放入运行队列中!
 3. 进程PCB在runqueue, 就是R, 不是这个进程正在运行, 才是运行状态
 4. 不要只意味, 你的进程只会等待(占用)CPU资源, 你的进程, 也可能随时随地, 要外设资源!!!
 5. 所谓的进程不同的状态, 本质是进程在不同的队列中, 等待某种资源!
- #define R 1
- 状态-> 进程内部的属性-> task_struct -> int (1: run, 2: stop, 3: hup 4: dead...)

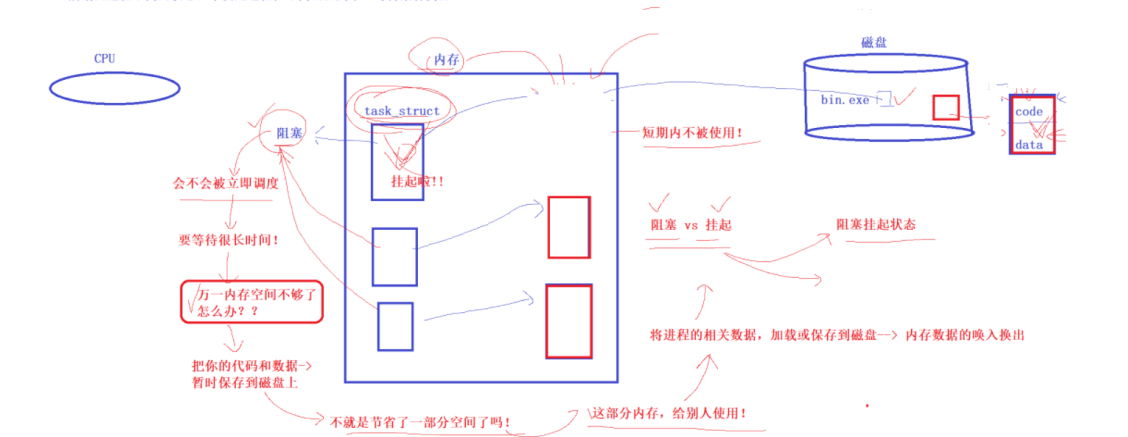
运行队列: 放要执行的进程

进程的状态: 进程内部的属性, 放在PCB里面。不同的场景, 不同的状态, 等待不同的资源。

阻塞状态: PCB放到不同的队列里面。等待着

挂起: 内核数据结构还在, code和data还在磁盘的

1. 一个CPU一个运行队列
 2. 让进程入队列, 本质: 将该进程的task_struct 结构体对象放入运行队列中!
 3. 进程PCB在runqueue, 就是R, 不是这个进程正在运行, 才是运行状态
 4. 不要只意味, 你的进程只会等待(占用)CPU资源, 你的进程, 也可能随时随地, 要外设资源!!!
 5. 所谓的进程不同的状态, 本质是进程在不同的队列中, 等待某种资源!
- #define R 1
- 状态-> 进程内部的属性-> task_struct -> int (1: run, 2: stop, 3: hup 4: dead...)



1. 什么叫做运行 ✓
2. 什么叫做阻塞 ✓
3. 什么叫做挂起 ✓

挂起状态信息

运行队列 (run queue)

1. 一个CPU一个运行队列。
2. 进程如队列, 该进程的PCB放入队列中。
3. 只要是在运行队列里面就是 R 状态。不是一定要真的被执行。

等待队列 (wait queue)

Linux 里通常不是“全局一个阻塞队列”,
而是“谁让你阻塞, 你就挂到谁的等待队列上”。

阻塞进程通常进入的是 各种 wait queue (等待队列), 而不是一个统一的“阻塞队列”。

CPU很快，外设很慢。

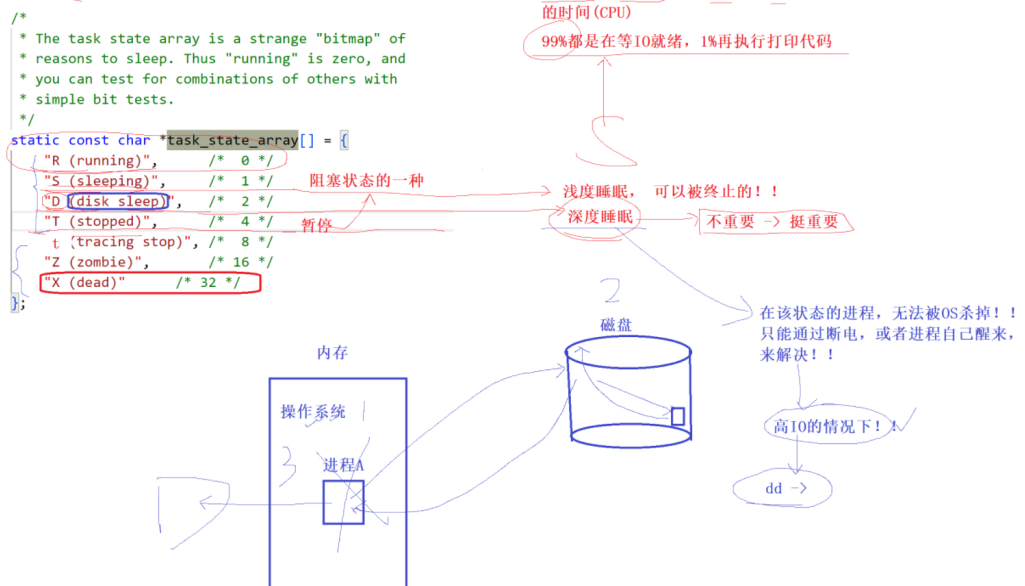
不同的队列，task_struct拿来拿去的。

进程不同的状态，就是进程等待不同的资源，等待 某种资源就绪。

Linux内核源代码：进程状态

```
/*
 * The task state array is a strange "bitmap" of
 * reasons to sleep. Thus "running" is zero, and
 * you can test for combinations of others with
 * simple bit tests.
 */
static const char * const task_state_array[] = {
    "R (running)", /* 0 */
    "S (sleeping)", /* 1 */
    "D (disk sleep)", /* 2 */
    "T (stopped)", /* 4 */
    "t (tracing stop)", /* 8 */
    "X (dead)", /* 16 */
    "Z (zombie)", /* 32 */
};
```

2. Linux是怎么做的？——> 具体的Linux操作系统的状态！



R运行状态 (running)：并不意味着进程一定在运行中，它表明进程要么是在运行中要么是在运行队列里面。

kill -19 暂停状态

kill -18 恢复暂停

kill -9 杀死进程

孤儿进程

父进程被杀掉，父进程的父进程回收资源了的。

父进程先退出，然后子进程被操作系统1号进程领养了。

1.孤儿进程一定存在

孤儿进程

父进程先退出。

给操作系统1号进程领养。

不领养，对应的僵尸，没有人能够回收了。

被领养的进程就是--孤儿进程。

如果是前台进程创建的子进程，如果孤儿了，后台进程了。

精灵进程，守护进程。

4.3 进程状态的code

4.3.1 R状态

```
#include<stdio.h>
#include<unistd.h>

int main()
{
    // 运行状态
    // 计算密集型的
    int a = 10;
    while(1)
    {
        a = 1 + 1;
    }
    return 0;
}
```

4.3.2 S状态

linux下的阻塞状态。

```
#include<stdio.h>
#include<unistd.h>
```

```
int main()
{
    // IO密集型的
    while(1)
    {
        printf("he11 sdf\n");
        sleep(1);
    }
    return 0;
}
```

4.3.3 T状态

kill -19 pid

kill -18 pid

```
root@lic lesson2]# C
root@lic lesson2]# vim io_intensive.c
root@lic lesson2]# sz io_intensive.c
root@lic lesson2]#
root@lic lesson2]# ./a.out

[]+ Stopped                               ./a.out
root@lic lesson2]#

root@lic ~]# cd 'linux/lesson/lesson2'
root@lic lesson2]# kill -19 23535
root@lic lesson2]#

19371 23644 23643 19371 pts/2    23643 R+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    23535 R+   0  0.23 ./a.out
19371 23648 23647 19371 pts/2    23647 R+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    23535 R+   0  0.24 ./a.out
19371 23651 23650 19371 pts/2    23650 R+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    23535 R+   0  0.25 ./a.out
19371 23654 23654 19371 pts/2    23654 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    23535 R+   0  0.26 ./a.out
19371 23659 23658 19371 pts/2    23658 R+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    23535 R+   0  0.27 ./a.out
19371 23663 23662 19371 pts/2    23662 R+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    23535 R+   0  0.28 ./a.out
15125 23535 23535 15125 pts/1    23666 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    23535 R+   0  0.29 ./a.out
19371 23670 23669 19371 pts/2    23669 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    23535 R+   0  0.30 ./a.out
19371 23673 23673 19371 pts/2    23673 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    23535 R+   0  0.31 ./a.out
19371 23678 23677 19371 pts/2    23677 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    23535 R+   0  0.32 ./a.out
19371 23682 23681 19371 pts/2    23681 R+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
15125 23535 23535 15125 pts/1    23685 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23690 23689 19371 pts/2    23689 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23693 23692 19371 pts/2    23692 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
15125 23535 23535 15125 pts/1    23696 S+   0  0.00 grep --color=auto a.out
19371 23701 23700 19371 pts/2    23700 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23705 23704 19371 pts/2    23704 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23709 23708 19371 pts/2    23708 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23712 23711 19371 pts/2    23711 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23716 23715 19371 pts/2    23715 S+   0  0.00 grep --color=auto a.out
```

```
root@lic lesson2]# ll
total 40
-rwxr-xr-x 1 root root 16384 Apr 19 10:02 a.out
-rw-r--r-- 1 root root 4092 Jan 25 21:46 class12.txt
-rw-r--r-- 1 root root 157 Apr 19 10:02 compute_intensive.c
-rw-r--r-- 1 root root 163 Apr 19 09:54 io_intensive.c
-rw-r--r-- 1 root root 28 Jan 25 21:46 makefile
-rw-r--r-- 1 root root 842 Apr 19 09:19 process.c
-rw-r--r-- 2 root root 4084 Jan 25 21:46 rwxer
root@lic lesson2]# vim compute_intensive.c
root@lic lesson2]# sz compute_intensive.c
root@lic lesson2]# ./a.out
C
root@lic lesson2]# ./a.out
C
root@lic lesson2]# C
root@lic lesson2]# vim io_intensive.c
root@lic lesson2]# sz io_intensive.c
root@lic lesson2]#
root@lic lesson2]# ./a.out

[]+ Stopped                               ./a.out
root@lic lesson2]#

Xshell 8 (Build 0090)
Copyright (C) 2024 NetSarang Computer, Inc. All rights reserved.
Type 'help' to learn how to use Xshell prompt.
[Ctrl]+[F]
Connecting to 8.140.211.98:22...
Connection established.
To escape to local shell, press 'Ctrl+Alt+J'.
[WARNING] The remote SSH server rejected X11 forwarding requests
Last login: Sun Apr 19 09:52:19 2026 from 117.172.173.43
Welcome to Alibaba Cloud Elastic Compute Service !

cd 'linux/lesson/lesson2'
root@lic ~]# cd 'linux/lesson/lesson2'
root@lic lesson2]# kill -19 23535
root@lic lesson2]# kill -18 23535
root@lic lesson2]#

19371 23852 23852 19371 pts/2    23852 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23857 23856 19371 pts/2    23856 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23861 23860 19371 pts/2    23860 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23865 23864 19371 pts/2    23864 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23869 23868 19371 pts/2    23868 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23872 23871 19371 pts/2    23871 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23876 23875 19371 pts/2    23875 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23880 23879 19371 pts/2    23879 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23884 23883 19371 pts/2    23883 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23888 23887 19371 pts/2    23887 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23892 23891 19371 pts/2    23891 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23896 23894 19371 pts/2    23894 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23899 23898 19371 pts/2    23898 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23903 23902 19371 pts/2    23902 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23907 23906 19371 pts/2    23906 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23911 23910 19371 pts/2    23910 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23914 23913 19371 pts/2    23913 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23918 23917 19371 pts/2    23917 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 T   0  0.32 ./a.out
19371 23922 23921 19371 pts/2    23921 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 R   0  0.33 ./a.out
19371 23926 23925 19371 pts/2    23925 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 R   0  0.34 ./a.out
19371 23930 23929 19371 pts/2    23929 R+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 R   0  0.35 ./a.out
19371 23933 23932 19371 pts/2    23932 R+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 R   0  0.36 ./a.out
19371 23937 23936 19371 pts/2    23936 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 R   0  0.37 ./a.out
19371 23941 23940 19371 pts/2    23940 R+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 R   0  0.38 ./a.out
19371 23945 23944 19371 pts/2    23944 R+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 R   0  0.40 ./a.out
19371 23949 23948 19371 pts/2    23948 R+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 R   0  0.41 ./a.out
19371 23953 23952 19371 pts/2    23952 S+   0  0.00 grep --color=auto a.out
15125 23535 23535 15125 pts/1    15125 R   0  0.42 ./a.out
19371 23956 23955 19371 pts/2    23955 R+   0  0.00 grep --color=auto a.out
```

恢复变成后台进程了的。

后台进程，只能kill -9 杀了的。

4.3.4 t状态

调试的时候就是

4.3.5 x状态无法查看

dead死亡状态。

4.3.6 D深度睡眠

一般都是在高IO场景的，无法被终止的。

4.3.7 z僵尸状态

僵尸状态是一个问题的。

进程退出都需要被回收的，linux设计哲学的。

10. 一句话总结

你这句话：

Linux 设计哲学，进程退出需要被父进程回收

总体是对的，但更准确的说法是：

Linux/Unix 为了让父进程获取子进程的退出状态，不会在子进程结束时立刻彻底删除它，而是先让它进入僵尸态，等待父进程通过 `wait/waitpid` 回收；如果父进程先退出，则由 1 号进程接管并回收。

为什么会有僵尸状态 -- 感性 -- 录制中00:24:51

进程被创建出来 --- 完成任务 --- 1. 要知道它完成的如何？ 2. 可以不关心结果

进程退出的时候，不能立即释放该进程对应的资源！
保存一段时间，让父进程或者OS来进行读取！

僵尸状态，是一个问题 -- 今天我们不解决

进程退出了，但是没有被回收(父进程、OS) -> 创建子进程，让父进程不要退出，而且什么都不做，让子进程正常退出！

子进程退出，等待父进程回收它的资源。


```
}
```

任何系统调用，都要先查“成功返回什么，失败返回什么，失败后 `errno` 表示什么”。

系统调用的健壮性 = 正确判断返回值 + 理解返回值语义 + 处理 `errno` + 做好资源清理。

4.4 孤儿进程

父进程先退出了的

父进程先退出 录制中01:01:20

1. 这种现象是一定存在
2. 子进程会被操作系统领养 -- 1号进程
3. 为什么要这么干？如果不领养，那么子进程退出的时候，对应的僵尸，便没有人能回收了！
4. 被领养的进程 --- 孤儿进程
5. 如果是前台进程创建的子进程，如果孤儿了，会自动变成后台进程

4.5 精灵进程

守护进程

5 优先级

了解范畴

了解范畴

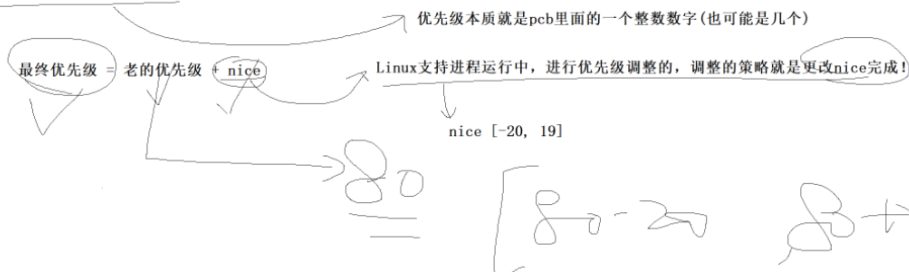
了解范畴

`ps -l`

`ps -al`

进程优先级 录制中01:05:56 了解范畴

1. 什么叫做优先级？
能还是不能 能，先做还是后做的问题
2. 为什么会存在优先级？
为什么会存在优先级？ 因为资源太少了！
3. Linux优先级特点 -- 很快



1. 什么叫做优先级

优先级是有执行的权利，只不过是先完成后完成的过程。 权限是谁可以做什么的问题

2. 为什么会存在优先级

为什么存在优先级？ 因为资源太少了。

3. Linux的优先级

优先级本质就是PCB里面的整数。

linux支持进程优先级调整的

```
pri 默认80
ni [-20, 19]
优先级的取值范围[60,99]
```

数字越小，优先级越高。

top r 输入pid [-20, 19]

每次都是默认的：80 +- ni

其它概念

其他概念

- 竞争性: 系统进程数目众多，而CPU资源只有少量，甚至1个，所以进程之间是具有竞争属性的。为了高效完成任务，更合理竞争相关资源，便具有了优先级
- 独立性: 多进程运行，需要独享各种资源，多进程运行期间互不干扰
- 并行: 多个进程在多个CPU下分别，同时进行运行，这称之为并行
- 并发: 多个进程在一个CPU下采用进程切换的方式，在一段时间之内，让多个进程都得以推进，称之为并发

竞争性：资源太少了，想运行就得竞争。

独立性：QQ退出了，不会影响微信的 野指针模拟实现

并行：多个CPU执行，多个 进程。 一个计算机 运行多个进程。 CPU的核数。

CPU = 工厂
CPU 核心 = 工人
任务 = 要处理的工作

1 核 CPU = 1 个工人
2 核 CPU = 2 个工人
4 核 CPU = 4 个工人
8 核 CPU = 8 个工人

并发：多个进程轮流到CPU执行。 看起来，大家都在执行。 时间片轮转的策略。进程不断切换。 不要用你的速度衡量CPU的速度。

4 核 8 线程
6 核 12 线程
8 核 16 线程
16 核 32 线程

概念	含义
核心数	CPU 里真正的物理计算核心数量
线程数	CPU 通过超线程技术暴露出来的逻辑执行单元数量

CPU任何一个时刻只能执行一个进程。

CPU 核数表示 CPU 里面有多少个物理计算核心；核心越多，越适合同时运行多个任务或多线程程序，但单线程程序不一定能充分利用多个核心。

进程切换：重点

pc(eip): 当前正在执行指令的下一条指令。

CPU：1取指令，2分析指令，3执行指令

当我们的进程在运行的时候，一定会产生很多的临时数据。这份数据属于当前进程！！

CPU虽然只有一套寄存器硬件，寄存器里面保存的数据，是属于当前进程的。

寄存器硬件 != 寄存器内的数据。

进程在运行的时候占CPU，进程不是占有到进程结束！！(while(1)) 死循环，那么就不能执行其它指令了。

进程在运行的时候都有自己的时间片，(离开学校，保留学籍的过程，上下文保护。)(回到学校，恢复学籍，上下文恢复。)

保留就是为了恢复。

寄存器(寄存，寄存，寄存进程的临时数据。)。

上下文：寄存器里面的数据

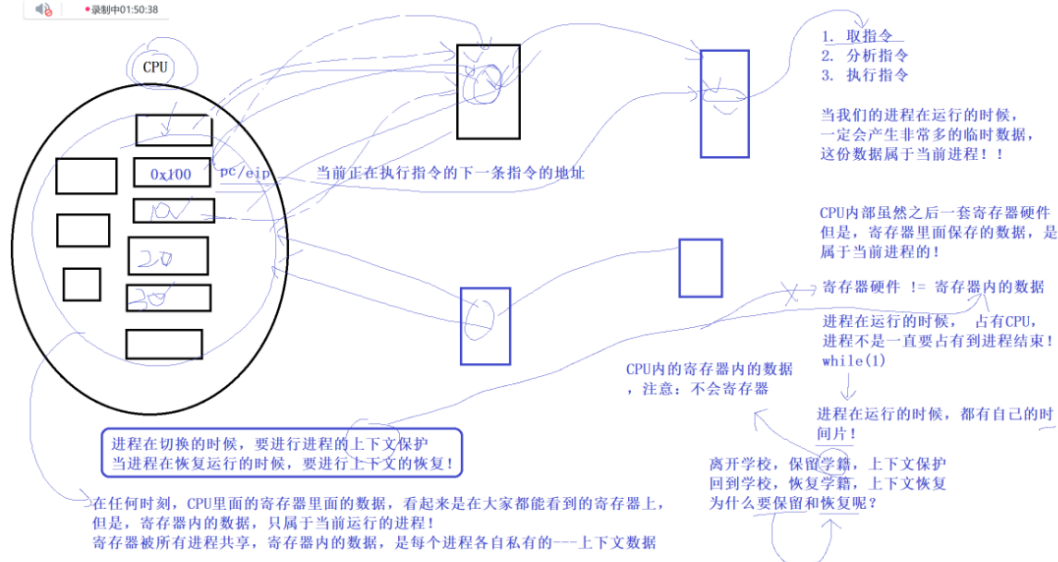
进程切换的时候， 要进程上下文保护。

当进程恢复的时候，要进程上下文恢复。

上下文数据：进程的上下文主要保存在该进程的“进程控制块”的数据结构中，而PCB通常存在于操作系统内核空间的内存里。

切换对线程的同步和互斥有助于理解的。

寄存器 被所有进程共享



难学的东西是知识，简单的东西是信息。

总结

进程切换的本质，是 CPU 从一个进程的“执行上下文”切换到另一个进程；因此，凡是与“继续正确执行”相关的状态，都必须被保存和恢复。

类别	保存 / 恢复内容	说明
CPU 上下文	PC、通用寄存器、SP、FLAGS	核心
内核栈	栈指针、栈内容	每进程独立
地址空间	页表基址 (CR3)	切换虚拟内存
PCB	状态、优先级、时间片	调度依据
信号	pending / mask	语义一致
FPU / SIMD	浮点与向量寄存器	延迟保存
TLB	刷新或标记	架构相关

复习

僵尸进程是一个进程的状态

孤儿进程，1号进程领养的。

进程优先级，进程调度的先后顺序。基础顺序80，调整顺序 -20,19。综合60-99。

进程切换，上下文保存，上下文恢复。寄存器：给 CPU 提供最快速的数据暂存和操作位置。CPU 芯片内部用于保存二进制数据的高速存储单元。

寄存器是 CPU 内部的高速临时存储单元，专门用来保存当前指令执行时最直接、最关键的数据、地址和状态。

上行文保存在：PCB里面的。

6环境变量

初始环境变量

不同的环境变量解决不同的问题的。

env

printenv

在 Linux 中，**环境变量 (environment variables)** 是操作系统中用于存储系统运行环境配置信息的一种 **键值对 (key-value pair)**，其主要作用是：用来影响进程的行为，例如查找命令路径、设定语言、用户目录、库路径等。

Linux的指令也是可执行程序。

切入点

要执行一个基本指令(程序)，先找到这个程序。

echo \$PATH

echo \$HOME

添加环境变量

export PATH=\$PATH:/home/lic/lesson11/

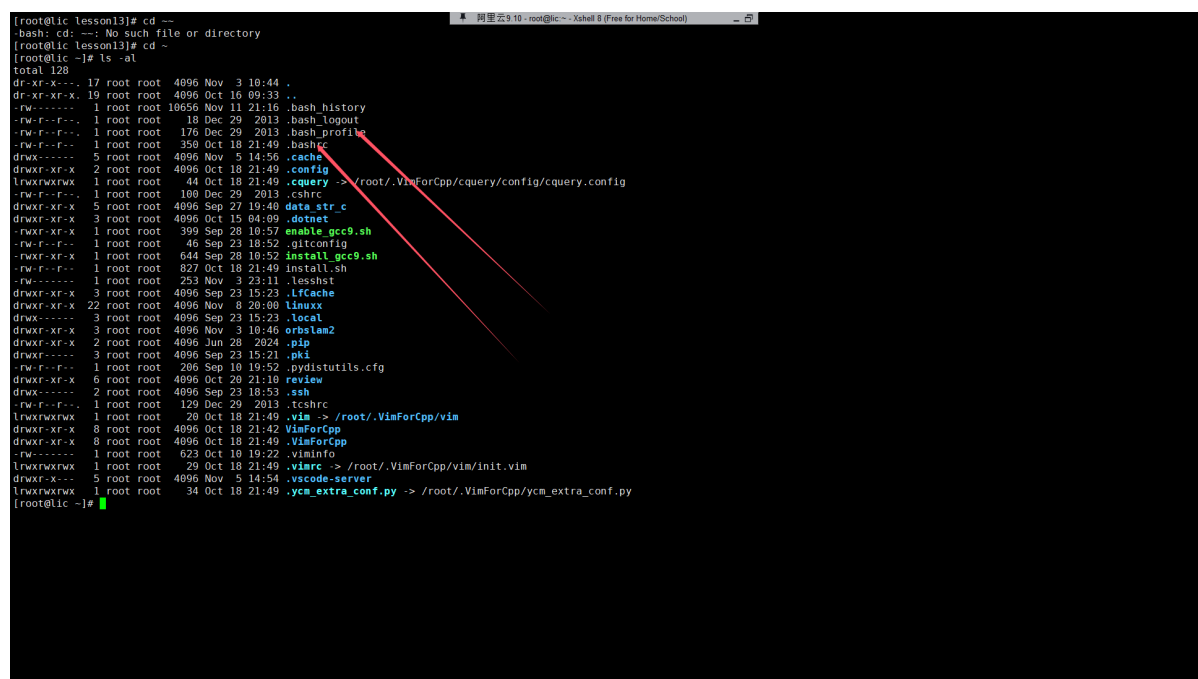
which本质就是根据环境变量PATH查找数据的。

Linux的环境变量配置：

启动一个shell 就会执行一次配置文件的。

/etc/bashrc

用户



```
[root@lic lesson13]# cd ~
-bash: cd: ~: No such file or directory
[root@lic lesson13]# cd ~
[root@lic ~]# ls -al
total 128
drwxr-xr-x. 17 root root 4096 Nov  3 18:44 .
drwxr-xr-x. 19 root root 4096 Oct 16 09:33 ..
-rw-r--r--. 1 root root 10656 Nov 11 21:16 .bash_history
-rw-r--r--. 1 root root 18 Dec 29 2013 .bash_logout
-rw-r--r--. 1 root root 176 Dec 29 2013 .bash_profile
-rw-r--r--. 1 root root 350 Oct 18 21:49 .bashrc
drwx----- 5 root root 4096 Nov  5 14:56 .cache
drwxr-xr-x. 2 root root 4096 Oct 18 21:49 .config
lrwxrwxrwx. 1 root root 44 Oct 18 21:49 cquery -> /root/.VimForCpp/cquery/config/cquery.config
-rw-r--r--. 1 root root 100 Dec 29 2013 .cshrc
drwxr-xr-x. 5 root root 4096 Sep 27 19:40 data_str.c
drwxr-xr-x. 3 root root 4096 Oct 15 04:09 .dotnet
-rwxr-xr-x. 1 root root 399 Sep 28 10:57 enable_gcc9.sh
-rw-r--r--. 1 root root 46 Sep 23 18:52 .gitconfig
-rwxr-xr-x. 1 root root 644 Sep 28 10:52 install_gcc9.sh
-rw-r--r--. 1 root root 827 Oct 18 21:49 install.sh
-rw-r--r--. 1 root root 253 Nov  3 23:11 .lessht
drwxr-xr-x. 3 root root 4096 Sep 23 15:23 .ifcache
drwxr-xr-x. 22 root root 4096 Nov  8 20:00 linuxx
drwx----- 3 root root 4096 Sep 23 15:23 .local
drwxr-xr-x. 3 root root 4096 Nov  3 10:46 orbslam2
drwxr-xr-x. 2 root root 4096 Jun 20 2024 .pip
drwx----- 3 root root 4096 Sep 23 15:21 .pkg
-rw-r--r--. 1 root root 206 Sep 10 19:52 .pydistutils.cfg
drwxr-xr-x. 6 root root 4096 Oct 20 21:10 review
drwx----- 2 root root 4096 Sep 23 18:53 .ssh
-rw-r--r--. 1 root root 129 Dec 29 2013 .tcshrc
lrwxrwxrwx. 1 root root 20 Oct 18 21:49 .vim -> /root/.VimForCpp/vim
drwxr-xr-x. 8 root root 4096 Oct 18 21:42 VimForCpp
drwxr-xr-x. 8 root root 4096 Oct 18 21:49 .VimForCpp
-rw-r--r--. 1 root root 623 Oct 10 19:22 .viminfo
lrwxrwxrwx. 1 root root 29 Oct 18 21:49 .vimrc -> /root/.VimForCpp/vim/init.vim
drwxr-xr-x. 5 root root 4096 Nov  5 14:54 .vscode-server
lrwxrwxrwx. 1 root root 34 Oct 18 21:49 .ycm_extra_conf.py -> /root/.VimForCpp/ycm_extra_conf.py
[root@lic ~]#
```

系统配置

/etc/bashrc

```
[root@lic etc]# ll
total 1408
-rw-r--r-- 1 root root      16 Jun 28 2024 adjtime
-rw-r--r-- 1 root root    1529 Apr  1 2020 aliases
-rw-r--r-- 1 root root  12288 Jun 28 2024 aliases.db
drwxr-xr-x 2 root root    4096 Jun 28 2024 alternatives
-rw-r----- 1 root root     541 May 16 2023 anacrontab
-rw-r--r-- 1 root root      55 Aug  8 2019 asound.conf
-rw-r--r-- 1 root root       1 May 18 2022 at.deny
drwxr-xr-x 3 root root    4096 Jun 28 2024 audisp
drwxr-xr-x 3 root root    4096 Jun 28 2024 audit
drwxr-xr-x 2 root root    4096 Sep 28 10:54 bash_completion.d
-rw-r--r-- 1 root root   2853 Apr  1 2020 bashrc
drwxr-xr-x 2 root root    4096 Dec  7 2023 binfmt.d
-rw-r--r-- 1 root root     37 May 21 2024 centos-release
-rw-r--r-- 1 root root     51 May 21 2024 centos-release-upstream
drwxr-xr-x 2 root root    4096 Oct 13 2020 chkconfig.d
-rw-r--r-- 1 root root    1507 Jun 28 2024 chrony.conf
-rw-r----- 1 root chrony   481 Aug  8 2019 chrony.keys
drwxr-xr-x 4 root root    4096 Jun 28 2024 cloud
drwxr-xr-x 2 root root    4096 Jun 28 2024 connttrackd
drwxr-xr-x 2 root root    4096 Jun 28 2024 cron.d
drwxr-xr-x 2 root root    4096 Jun 28 2024 cron.daily
-rw-r----- 1 root root       0 May 16 2023 cron.deny
```

echo \$HOME

复习

父进程

子进程：处于僵尸状态，为了让自己状态被父进程知道。

孤儿进程。操作系统的一号进程领养。

孤儿进程会变成后台进程。

优先级

```
pri 80
ni [-20, 19]
```

进程获得资源的先后顺序

优先级：资源的有限性。

cpu里面的大量寄存器

上下文保护

上下文恢复

进程切换

并行：多cpu执行多个进程

并发：进程切换。看起来所有的进程都在执行。（大量上下文的切换和上下文恢复）

时间片

进程的临时数据保存着PCB里面和运行队列里面。

CPU的功能

取指令

分析指令

执行指令

c/c++代码----->编译链接----->CPU的指令

PATH

是可执行指令的路径

shell也是一个进程。

进程优先级一般不需要调整，除非你特别了解。或者你真的需要调整。

进程切换，CPU轮转的临时数据，在寄存器里面的数据(进程的上下文)，上下文保护，上下文切换。

CPU:取指令，分析指令，执行指令。

进程切换

时间片轮转。执行进程。

并发：每个进程在时间片跑跑跑。

时间片单位，

CPU---PCB---Code&Data--address--reall address

例子：学籍保留

su -重新登录

su 切换用户

不同的环境变量，使用不同的场景。

user:环境变量的意义，可以表示当前那个用户在使用Linux

```
// #include <stdlib.h>
// char *getenv(const char *name);

#include <stdio.h>
#include <stdlib.h>
int main()
{
    printf("%s \n", getenv("HOME"));
    printf("%s \n", getenv("USER"));
    printf("%s \n", getenv("PWD"));
    printf("%s \n", getenv("PATH"));
    return 0;
}
```

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define USER "USER"

int main()
{
    char* user = getenv(USER);
    if(strcmp(user, "root") == 0)
    {
        printf("user:%s\n", user);
    }
    else
```

```

{
    printf("权限不足\n");
}
return 0;
}

```

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define USER "USER"
#define MY_VAL "myval"

int main()
{

    char* myenv = getenv(MY_VAL);
    if(NULL == myenv)
    {
        printf("%s, not found\n", MY_VAL);
        return 1;
    }

    printf("%s:%s\n", MY_VAL, myenv);

    return 0;
}

```

USER 环境变量最大的意义，可以标识当前的使用Linux用户

\$./mycmd — bash就是一个系统进程，mycmd也会变成一个进程（fork），是bash的子进程 — 环境变量具有全局属性 — 是会被子进程继承下去（为什么要，为了不同的应用场景 — 让bash帮我找指令路径，身份认证）
 myval=1234567 — 本地变量 — 只会在当前进程（bash）内有效

环境变量具有全局属性，会被子进程继承下去，也方便子进程满足不同的场景。

unset, export.

ls指令，子进程继承，知道PWD, 然后就知道了。

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define USER "USER"
#define MY_VAL "myval"
#define MYPWD "PWD"
int main()
{

    printf("%s\n", getenv(MYPWD));

    return 0;
}

```

本地环境变量和全局环境变量

aaa=123

export aaa 导入全局环境变量

当前bash的全局环境变量，会被子进程继承下去的。

本地变量-- 只会在当前bash内有效的

aaa=123 定义局部环境

export aaa 设置全局环境变量

unset aaa 取消全局环境变量

export MYAVL=2222

ls 继承了PWD，所以不断切换目录，ls知道你当前在那个目录下面。

环境变量组织形式

环境变量是如何被子进程继承??

main函数参数和extern enviro

命令行参数

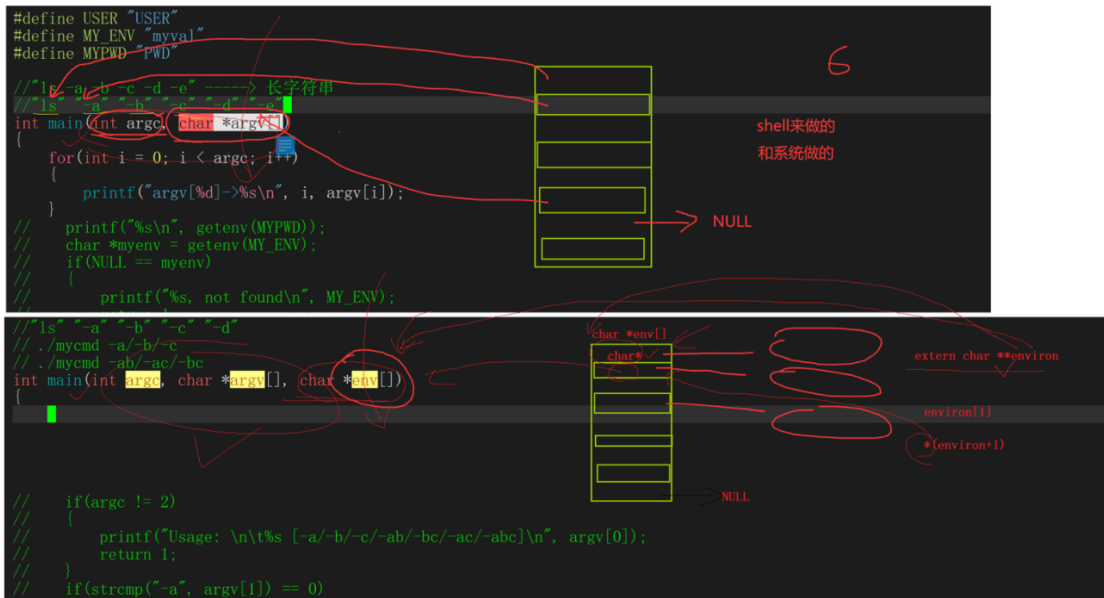
argc个数

argv内容

```
#include <stdio.h>
int main(int argc, char* argv[])
{
    for(int i = 0; i < argc; i++)
    {
        printf("argv[%d] : %s\n", i, argv[i]);
    }
    return 0;
}
```

```
[root@lic lesson14]# ./a.out asdf asdf
argv[0] : ./a.out
argv[1] : asdf
argv[2] : asdf
[root@lic lesson14]#
```

命令行参数



命令行选项

```
#include <stdio.h>
#include <string.h>

// ./mycmd -a -b -c
// ./mycmd -ab -bc -ac
// ./mycmd -abc

int main(int argc, char* argv[])
{
    if(argc != 2)
    {
        printf("Usage: \n%s [-a/-b/-c/-ab/-ac/-bc/-abc]\n", argv[0]);
        return 1;
    }

    if(strcmp("-a", argv[1]) == 0)
    {
        printf("功能a\n");
    }

    if(strcmp("-b", argv[1]) == 0)
    {
        printf("功能b\n");
    }

    if(strcmp("-c", argv[1]) == 0)
    {
        printf("功能c\n");
    }

    if(strcmp("-ab", argv[1]) == 0)
    {
        printf("功能ab\n");
    }
}
```

```

if(strcmp("-ac", argv[1]) == 0)
{
    printf("功能ac\n");
}

if(strcmp("-bc", argv[1]) == 0)
{
    printf("功能bc\n");
}
if(strcmp("-abc", argv[1]) == 0)
{
    printf("功能abc\n");
}
return 0;
}

```

环境变量表

```

#include <stdio.h>
#include <string.h>

int main(int argc, char* argv[], char* env[])
{
    for(int i = 0; env[i]; i++)
    {
        printf("%d : %s\n", i, env[i]);
    }
    return 0;
}

```

extern char** environ

```

#include <stdio.h>
#include <string.h>
#include <unistd.h>

extern char** environ;
int main()
{
    for(int i = 0; environ[i]; i++)
    {
        printf("%d : %s \n", i, environ[i]);
    }

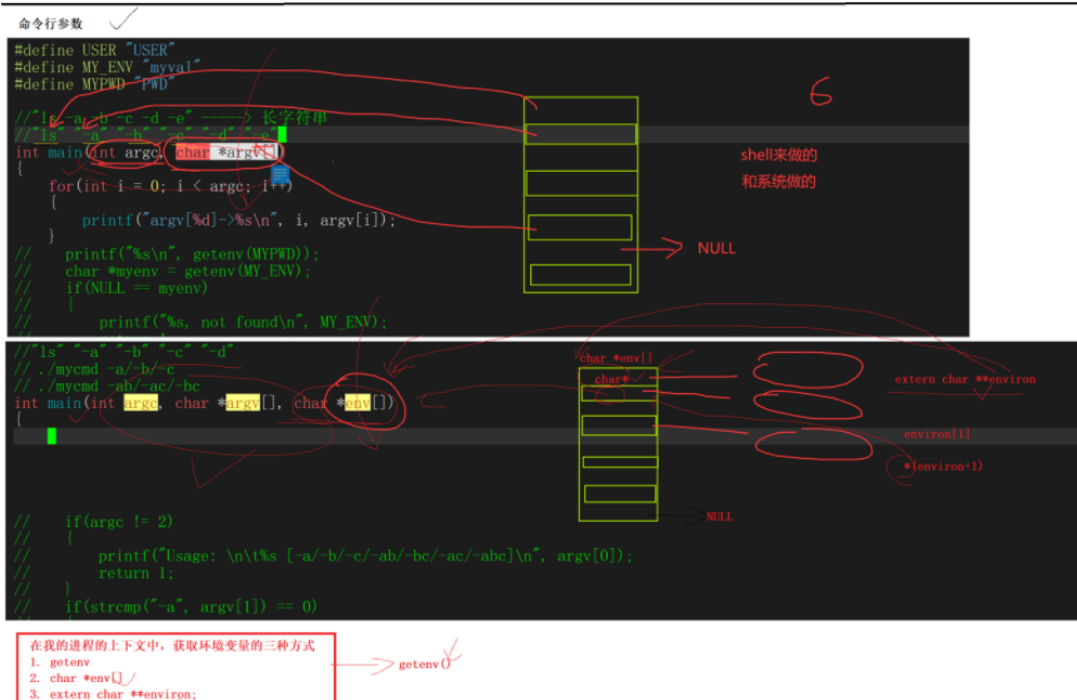
    return 0
}

```

getenv()

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    printf("%s \n", getenv("HOME"));
    printf("%s \n", getenv("USER"));
    printf("%s \n", getenv("PWD"));
    printf("%s \n", getenv("PATH"));
    return 0;
}
```



putenv()

```
#include <stdlib.h>
```

```
int putenv(char *string);
```

putenv不重要

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
#include <unistd.h>
```

```
int stat(const char *pathname, struct stat *statbuf);
```

内建指令

```
[whb@bite-alicloud lesson14]$ hello=8888
[whb@bite-alicloud lesson14]$ env | grep hello
[whb@bite-alicloud lesson14]$ set | grep hello
hello=8888
[whb@bite-alicloud lesson14]$ echo $hello
8888
```

echo 是一个命令吗? echo最后是一个子进程

这里为何能打印本地变量的值??

hello: 是一个本地变量, 不可以被子进程继承

复习

环境变量

命令行参数

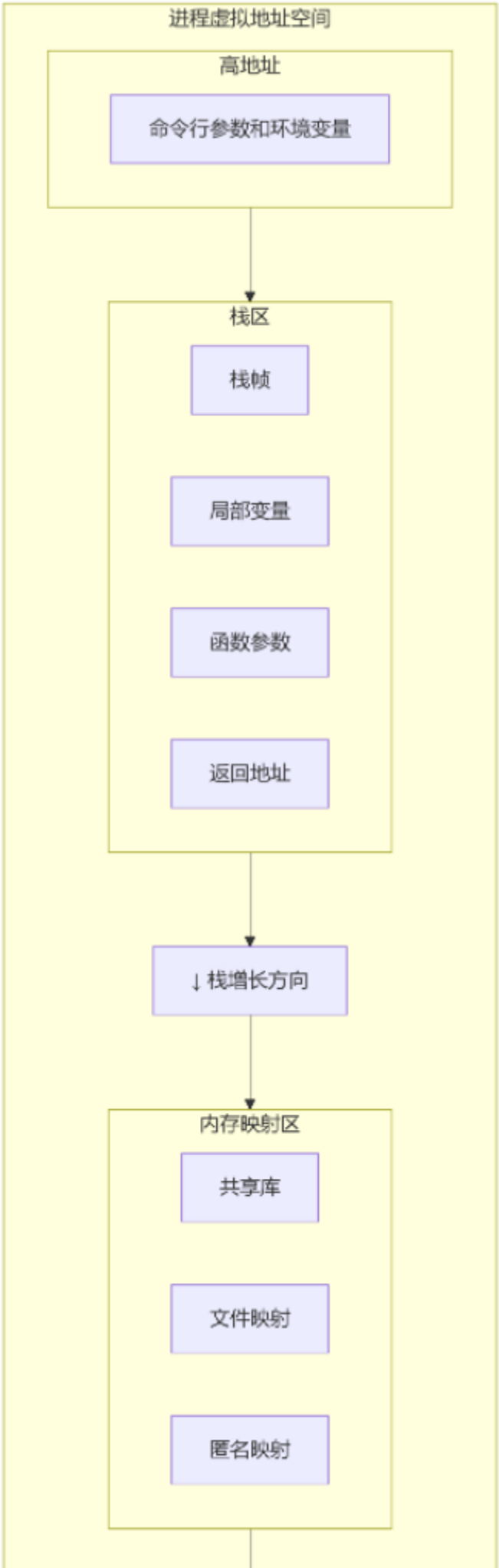
命令行参数表，环境变量表。

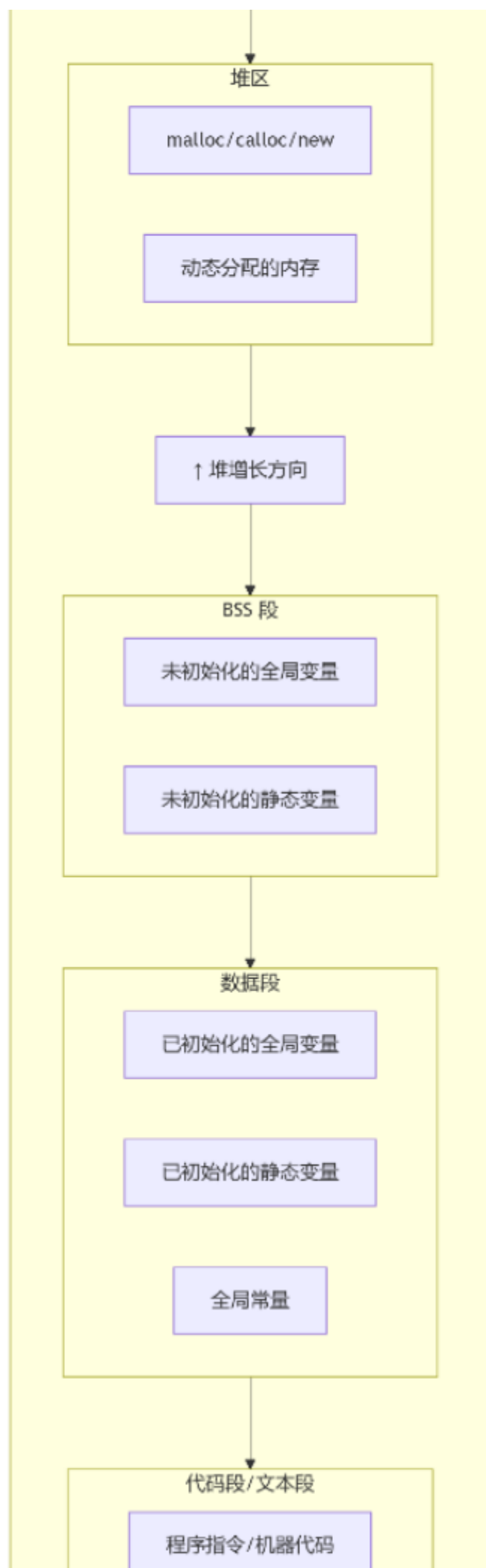
main函数的两张表

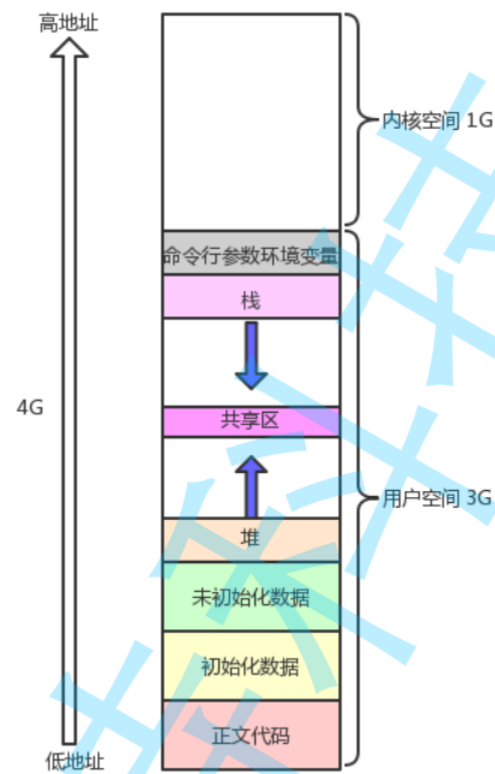
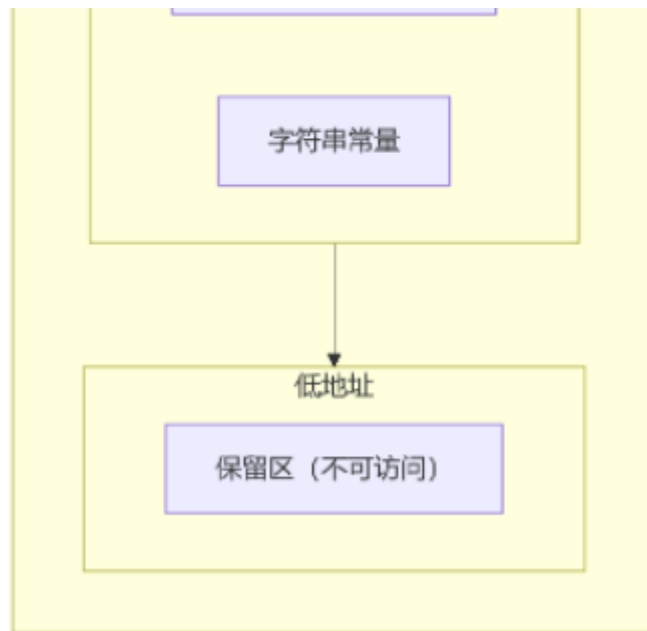
内建指令

虚拟地址空间。见见猪跑的

7程序地址空间







可是我们对他并不理解!


```

{
    printf("我是父进程、 g_val: %d, &g_val : %p \n", g_val, &g_val);
    sleep(1);
}
}

return 0;
}

```

```

[root@lic lesson14]# ./a.out
我是父进程、 g_val: 2, &g_val : 0x40404c
我是子进程、 g_val: 1, &g_val : 0x40404c
我是父进程、 g_val: 2, &g_val : 0x40404c
我是子进程、 g_val: 1, &g_val : 0x40404c
我是父进程、 g_val: 2, &g_val : 0x40404c
我是子进程、 g_val: 1, &g_val : 0x40404c
我是父进程、 g_val: 2, &g_val : 0x40404c
我是子进程、 g_val: 1, &g_val : 0x40404c
我是父进程、 g_val: 2, &g_val : 0x40404c
我是子进程、 g_val: 1, &g_val : 0x40404c
我是父进程、 g_val: 2, &g_val : 0x40404c
我是子进程、 g_val: 1, &g_val : 0x40404c
我是父进程、 g_val: 2, &g_val : 0x40404c
我是子进程、 g_val: 1, &g_val : 0x40404c
我是父进程、 g_val: 2, &g_val : 0x40404c
我是子进程、 g_val: 1, &g_val : 0x40404c
我是父进程、 g_val: 2, &g_val : 0x40404c
我是子进程、 g_val: 1, &g_val : 0x40404c

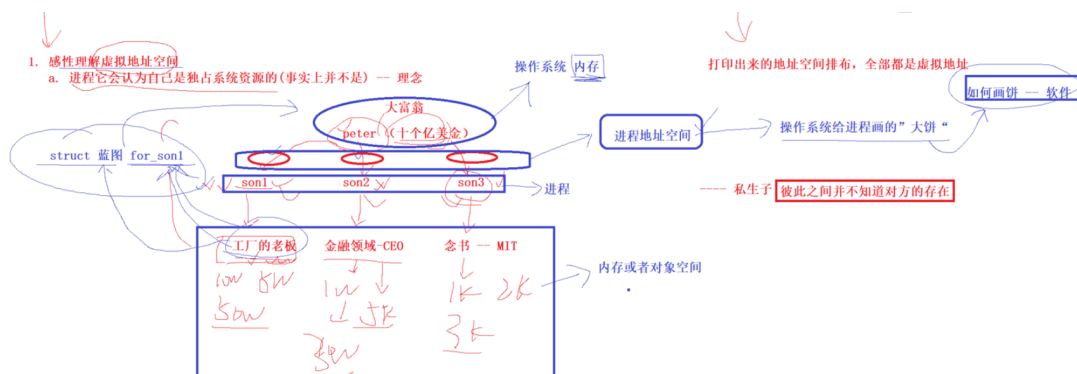
```

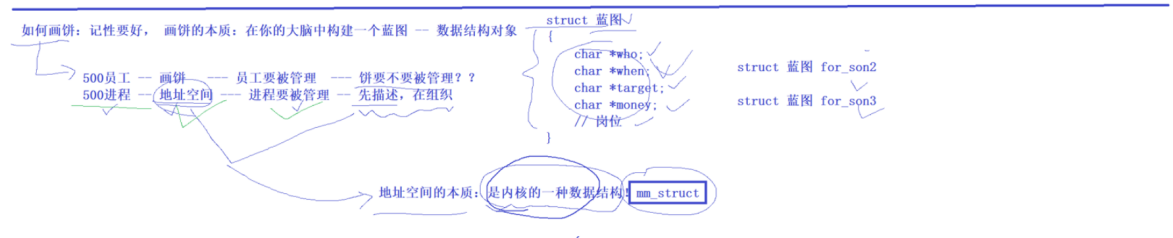
多进程读取同一个地址，怎么可能出现不同的结果？ 进程的独立性，同一个地址，不同的数据。

地址没变----》这里的地址，绝对不是物理地址---》曾经我们学习语言级别的地址，绝对不是对应的物理地址。---》虚拟地址（线性地址） c/c++打印出来的全部都是虚拟地址。

进程他会认为自己独占系统资源（实际上不是）。这是设计上的理念。

三son





复习

环境变量，全局属性，方便不同的场景使用

命令行参数，根据选项执行不同的功能。

mian函数表，环境变量表。推荐getenv()

echo 怎么能够打出局部变量

进程地址空间，是虚拟地址空间。

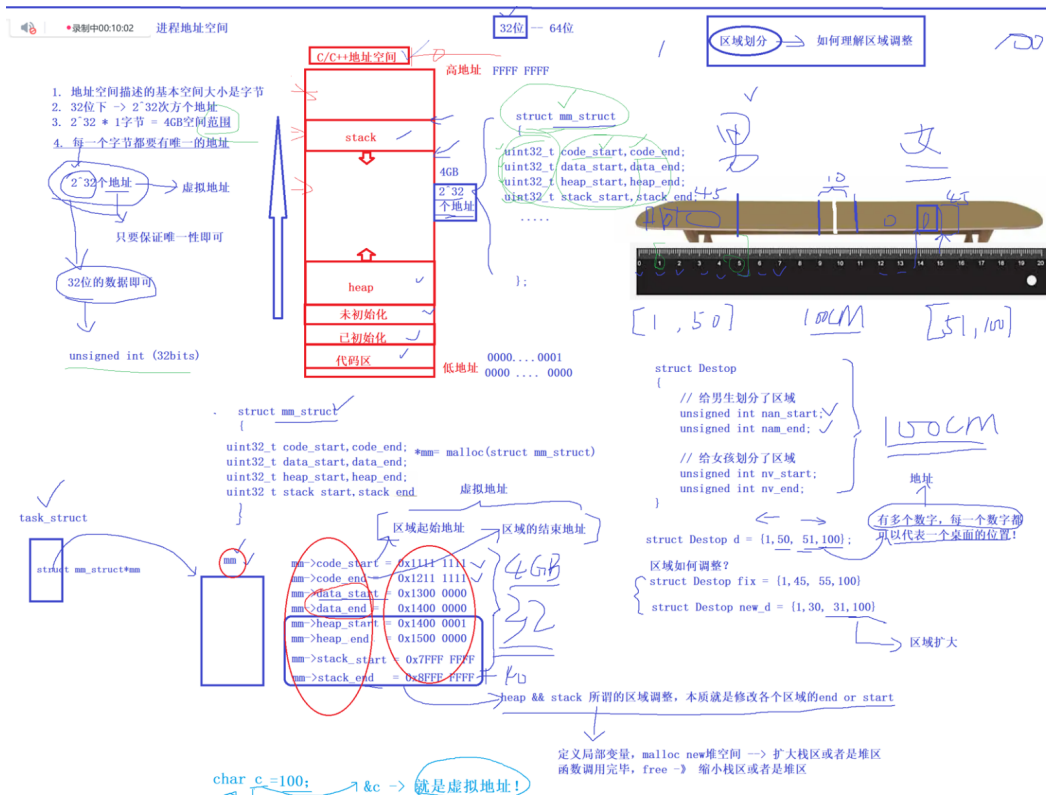
进程独立性，打印一个地址，然而不是不同的数据。

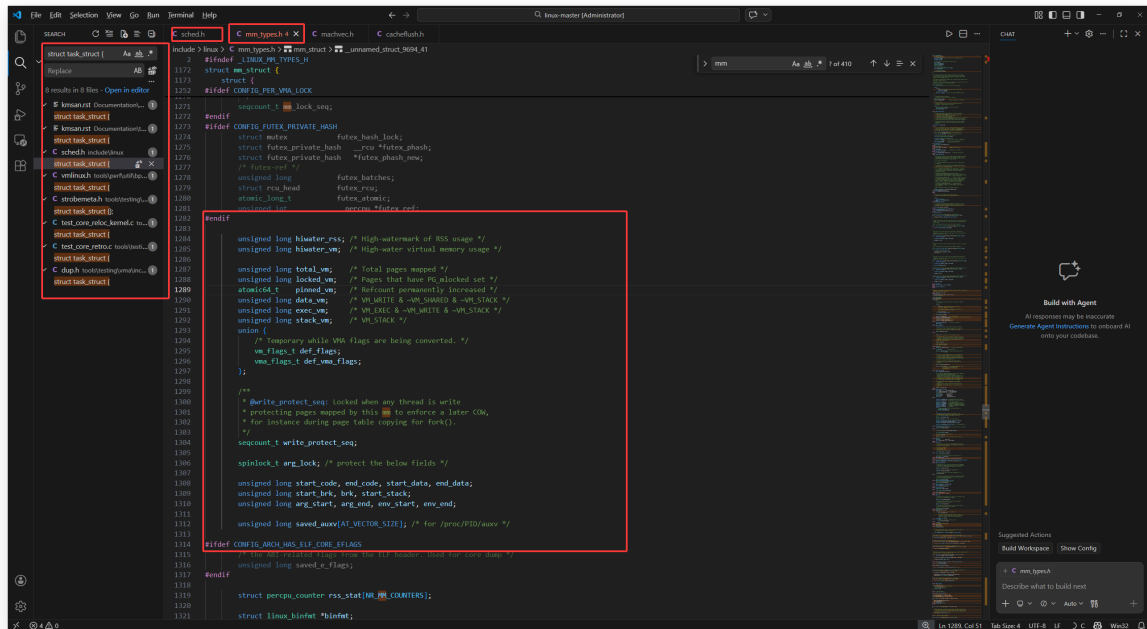
进程地址空间，就是一种数据结构。每一个地址空间管理起来。mm_struct

复习

区域划分

区域划分就是进程地址空间。





mm_struct{} 成员有哪些呢？

- 1.地址空间的描述的基本空间大小是字节
- 2.32位---2^32个地址
- 3.2^32 * 1字节 = 4GB的空间范围
- 4.每一个字节唯一的地址

2^32个地址保证唯一性就行了。

32位的数据即可

43 默认就是2^32次方个地址。

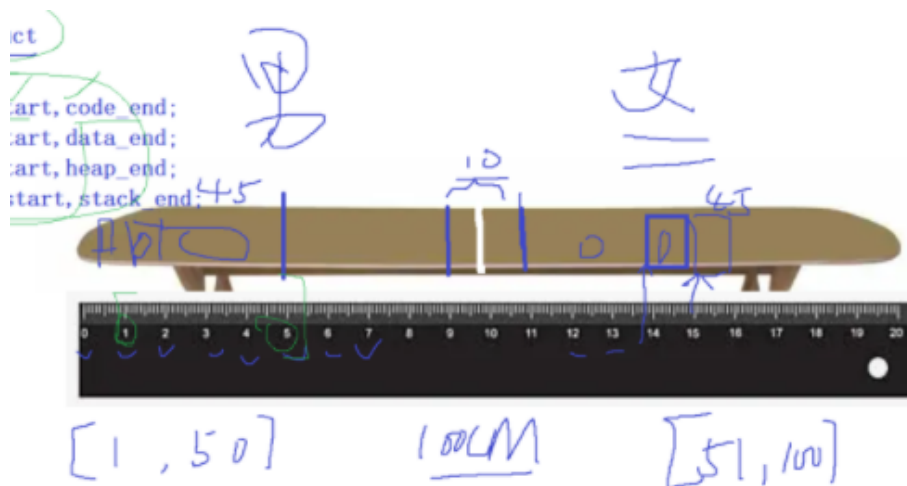
unsigned int(32bits)

struct mm_struct

```
{
    uint32_t code_start;
    uint32_t code_end;
    uint32_t data_start;
    uint32_t data_end;
    uint32_t heap_start;
    uint32_t heap_end;
    uint32_t stack_start;
    uint32_t stack_end;
};
```

[start, end]---[起始地址， 区域结束地址]

举个栗子



```

struct Destop
{
    // 给男生划分了区域
    unsigned int nan_start; ✓
    unsigned int nan_end; ✓

    // 给女孩划分了区域
    unsigned int nv_start;
    unsigned int nv_end;
}

struct Destop d = {1, 50, 51, 100};

```

100CM

地址

有多个数字，每一个数字都可以代表一个桌面的位置！

区域如何调整？

```

struct Destop fix = {1, 45, 55, 100}
struct Destop new_d = {1, 30, 31, 100}

```

区域扩大



[start, end]---[起始地址，区域结束地址]

struct mm_struct* p = (struct mm_struct*)malloc(sizeof(struct mm_struct));

```

-----无法修改-----
p->code_start = ;
p->code_end = ;
p->data_start = ;
p->data_end = ;

```

66 -----堆栈可以调整----- (本质就是修改end或者start)

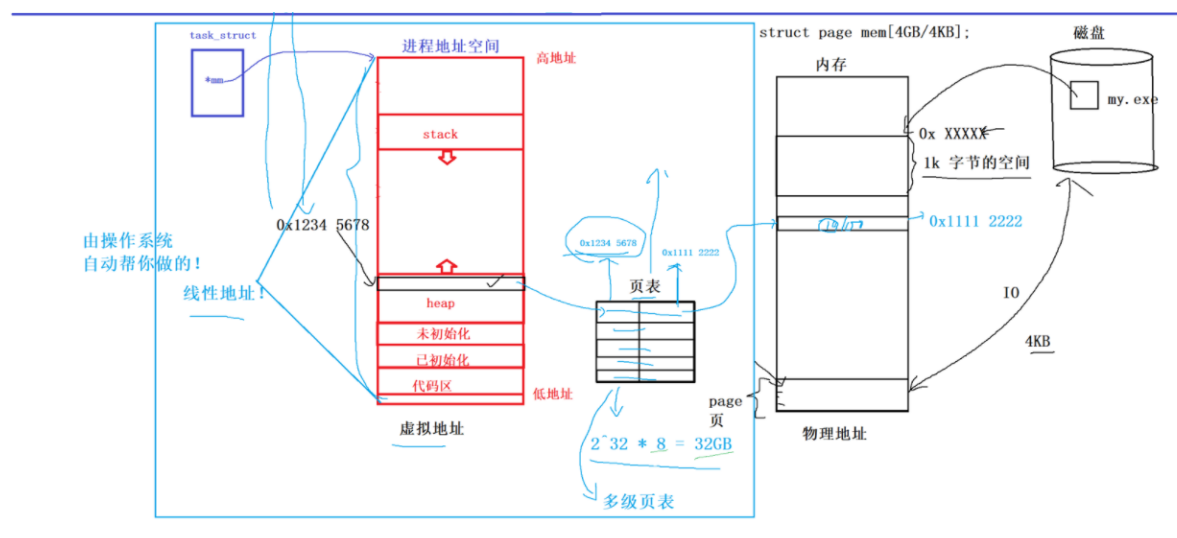
67 定义局部变量， malloc ,new---->扩大堆栈区域

68 函数调用完毕， free, delete-->缩小堆栈区域

地址

有多个数字，每一个数字都可以代表一个桌面的位置！

页表



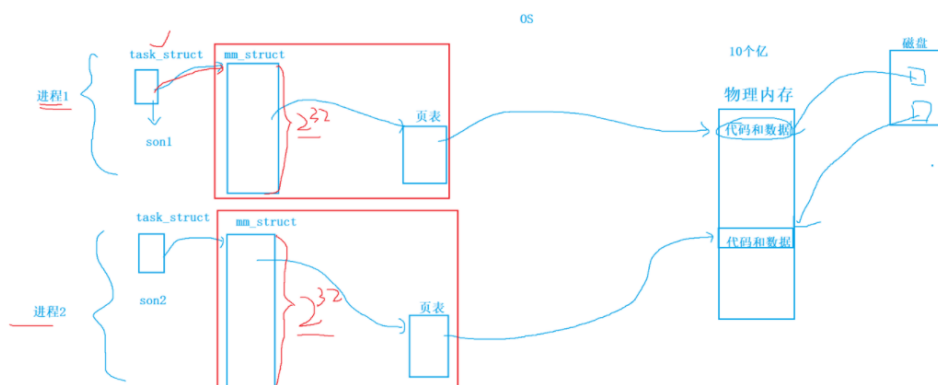
程序加载到内存里面

OS创建PCB--->里面有程序的信息。

内存IO一般是4KB为单位的。Page页

线性地址：数组就是线性地址。

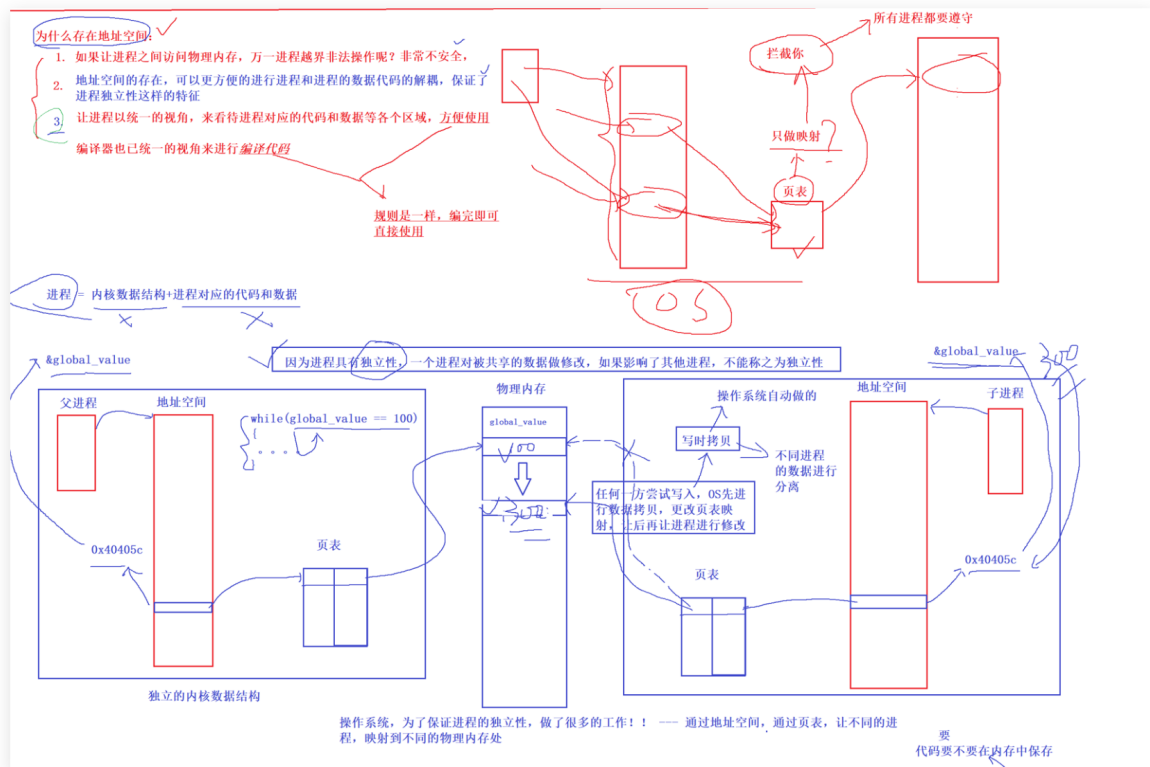
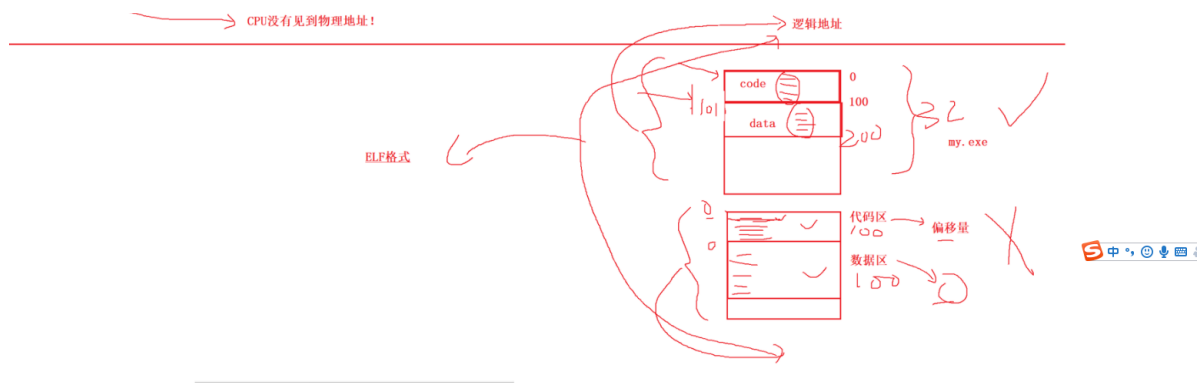
在进一步



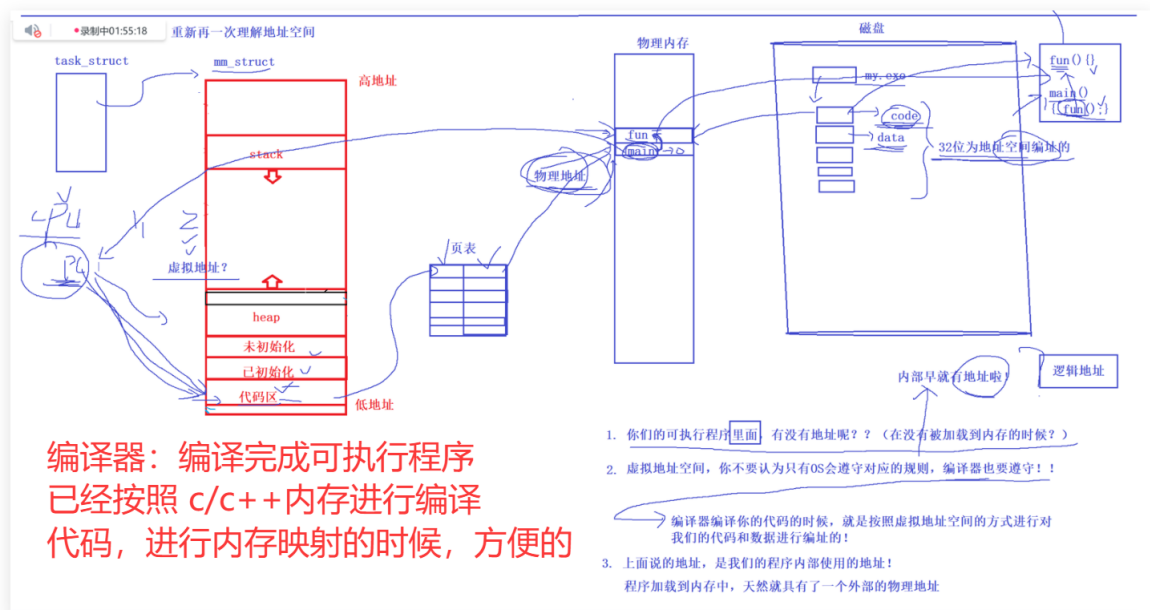
虚拟地址优点

如何理解地址空间呢？

- 1.如果让进程直接访问物理内存，万一进程越界非法操作呢？非常不安全呢！（页表不仅仅只做映射，还会检查映射是否安全呢）安全！所以进程都必须遵守的。安全的
 - 2.地址空间的存在可以更方便，进程和进程的数据代码的解耦，保证了进程的独立性的特征。写时拷贝。地址空间通过页表映射。
 - 3.让进程以同一的视角，来看待进程对应的代码和数据等各个区域，方便编译器也以统一的视角进行编译代码。规则一样的，编译完即可直接使用的。
- 编译代码的时候，使用不同的地址形式。



父子进程这么亲密都无法，相互干扰，何况其它进程呢？



链接库的时候，就是把他们的地址，编译到我的程序 里面去。

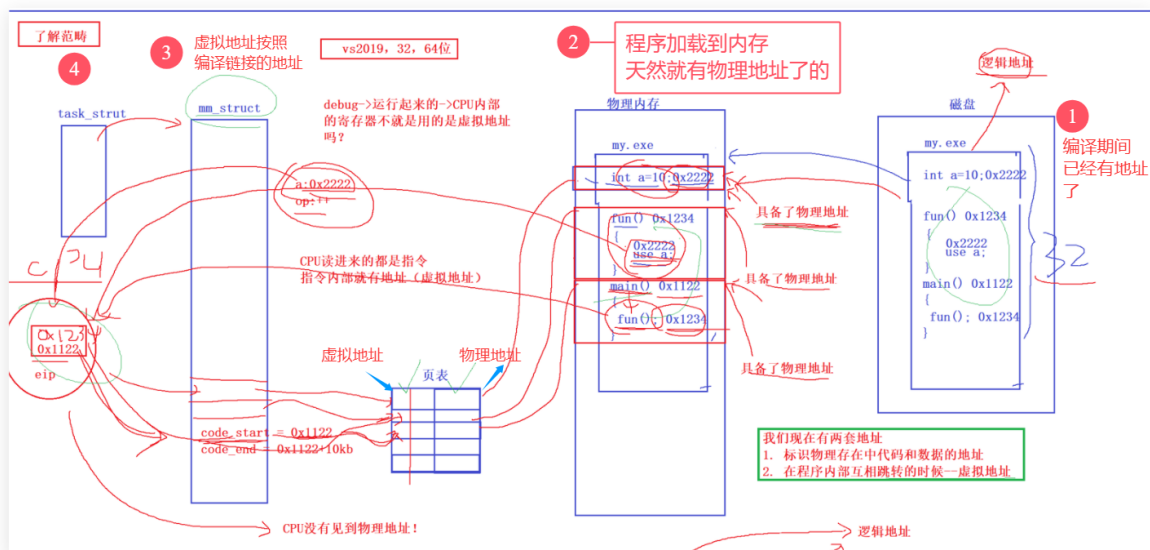
1.你的可执行程序，编译链接完成已经就存在地址了，逻辑地址的。

2.虚拟地址空间，不知OS会遵守对应的规则，编译器要遵守！！

编译器就是按照虚拟地址空间进行编译的。

3.程序加载到内存里面，天然就有物理地址了。

虚拟地址---就是你程序编译好的逻辑地址。



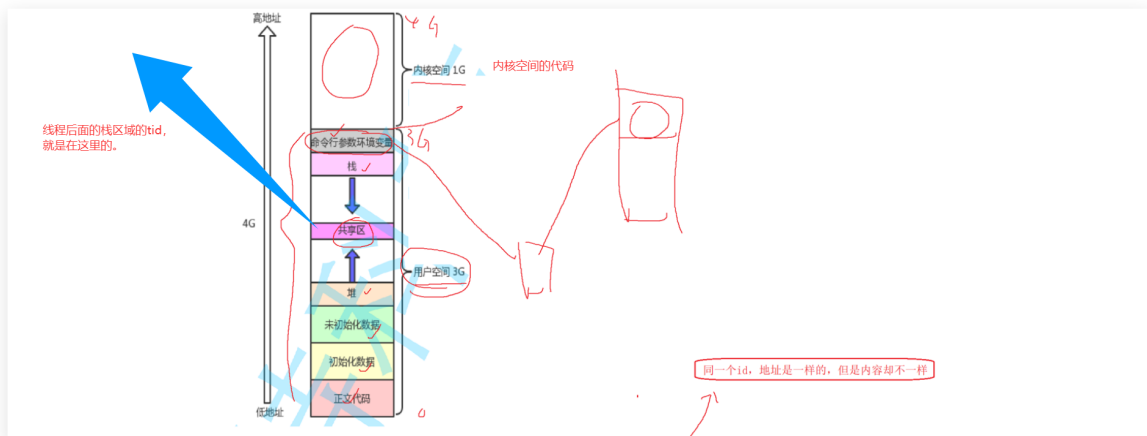
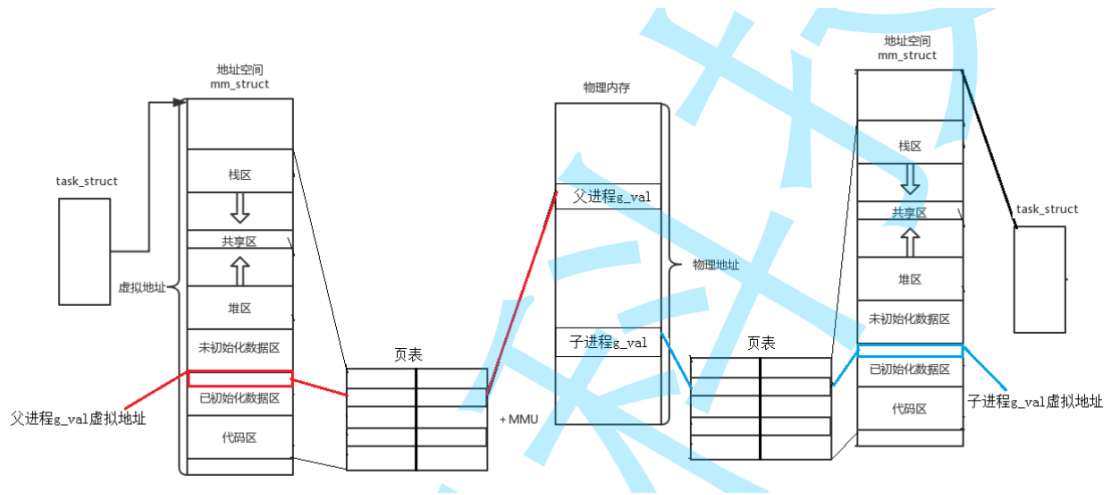
CPU读取的指令，存在地址。

地址

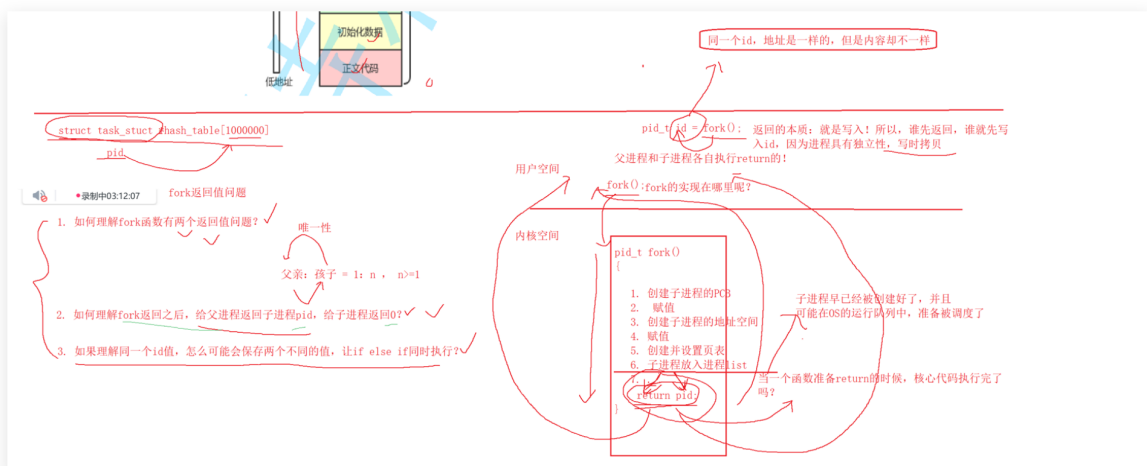
两种地址



file 指令



fork函数的问题



code

mian_table

```
#include <stdio.h>

int main(int argc, char* argv[], char* env[])
{
    for(int i = 0; i < argc; ++i)
    {
        printf("%d:%s\n", i, argv[i]);
    }

    printf("-----\n");

    for(int i = 0; env[i]; ++i)
    {
        printf("%d:%s\n", i, env[i]);
    }

    return 0;;
}
```

fork

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    pid_t id = fork();
    if(id == -1)
    {
        perror("fork");
        return 1;
    }

    if(id == 0)
    {
        while(1)
        {
            printf("子进程: %d,%d\n", getppid(), getpid());
            sleep(2);
        }
    }
    else
    {
        while(1)
        {
            printf("父进程: %d, %d\n", getppid(), getpid());
            sleep(2);
        }
    }
}
```

```
    return 0;
}
```

常见环境变量

```
#include <stdlib.h>
#include <stdio.h>

int main()
{
    printf("%s\n", getenv("USER"));
    printf("%s\n", getenv("HOME"));
    printf("%s\n", getenv("PATH"));
    printf("%s\n", getenv("LOGNAME"));
    printf("%s\n", getenv("PWD"));

    return 0;
}
```

本节重点

本节重点：

- 认识冯诺依曼系统
- 操作系统概念与定位
- 深入理解进程概念，了解PCB
- 学习进程状态，学会创建进程，掌握僵尸进程和孤儿进程，及其形成原因和危害
- 了解进程调度，Linux进程优先级，理解进程竞争性与独立性，理解并行与并发
- 理解环境变量，熟悉常见环境变量及相关指令，getenv/setenv函数
- 理解C内存空间分配规律，了解进程内存映像和应用程序区别，认识地址空间。
- 选学Linux2.6 kernel，O(1)调度算法架构

